ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG ĐIỆN – ĐIỆN TỬ

BÁO CÁO
ĐỒ ÁN THIẾT KẾ II

ĐỀ TÀI:

THIẾT KẾ HỆ THỐNG ĐO NHIỆT ĐỘ VÀ CẢNH BÁO NGƯỠNG SỬ DỤNG ATMEGA16

Giảng viên hướng dẫn:	ThS. Vũ Hồng Vinh	
Sinh viên thực hiện:	Mai Duy Hiếu	20223967
	Hoàng Quốc Hưng	20233758
Mã lớp học:	772642	
Nhóm thực hiện:	Nhóm 4	

Hà Nội, 11/2026

LỜI NÓI ĐẦU

Đề tài "Thiết kế hệ thống đo nhiệt độ và cảnh báo ngưỡng sử dụng ATmega16" thuộc học phần Đồ án Thiết kế II. Hệ thống đọc cảm biến DHT11, hiển thị dữ liệu lên LCD 1602 và bật 5 LED khi giá trị nhiệt độ nhận được đạt hoặc vượt ngưỡng cài đặt.

Nhóm vẽ lại sơ đồ nguyên lý từ tài liệu giảng viên cung cấp và thiết kế PCB trên cơ sở sơ đồ đó. Phần mềm gồm giao tiếp DHT11, cấu hình xung nhịp, vô hiệu hóa JTAG và điều khiển LCD, LED trên các cổng dùng chung. Thư viện `hunget_lcd.h` được xây dựng với hỗ trợ của công cụ sinh mã.

Ngưỡng 30°C được chọn làm mốc kiểm thử chức năng của mô hình; kết quả thực nghiệm tập trung vào hiển thị, cảnh báo ngưỡng và xử lý mất tín hiệu cảm biến.

Báo cáo đồ án được kết cấu thành 4 chương chính như sau:

- **Chương 1**: Giới thiệu ATmega16, kit AVR, cảm biến DHT11, LCD 1602 và khối LED.
- **Chương 2**: Trình bày yêu cầu thiết kế, lựa chọn linh kiện, sơ đồ, thiết kế PCB, kết nối phần cứng, lưu đồ và các thuật toán điều khiển, quy trình biên dịch và nạp firmware.
- **Chương 3**: Tổng hợp kết quả chức năng và giới hạn kiểm chứng thực nghiệm.
- **Chương 4**: Nêu kết luận, hạn chế và hướng phát triển.

Chúng em trân trọng cảm ơn thầy ThS. Vũ Hồng Vinh đã hướng dẫn thực hiện đồ án, cùng các thầy cô Trường Điện – Điện tử đã giảng dạy những kiến thức nền tảng. Nhóm mong nhận được góp ý để tiếp tục hoàn thiện mô hình và báo cáo.

Hà Nội, tháng 11 năm 2026

Sinh viên thực hiện

Mai Duy Hiếu

Hoàng Quốc Hưng

MỤC LỤC

LỜI NÓI ĐẦU i

DANH MỤC HÌNH VẼ iv

DANH MỤC BẢNG v

DANH MỤC TỪ VIẾT TẮT vi

1 GIỚI THIỆU TỔNG QUAN PHẦN CỨNG VÀ LINH KIỆN 1

1.1 Vi điều khiển ATmega16 1

1.1.1 Tổng quan kiến trúc và đặc tính kỹ thuật 1

1.1.2 Cấu trúc các cổng vào/ra và chức năng trong đồ án 2

1.1.3 Xung nhịp hệ thống và phân tích xung đột ngoại vi JTAG 3

1.2 Mạch Kit thực hành vi điều khiển họ AVR 4

1.3 Cảm biến số nhiệt độ và độ ẩm DHT11 5

1.3.1 Cấu tạo và nguyên lý hoạt động 5

1.3.2 Giao thức truyền thông đơn tuyến Aosong 1-Wire 6

1.4 Màn hình hiển thị tinh thể lỏng LCD 1602 8

1.5 Khối cảnh báo và chỉ thị LED 9

2 THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG 11

2.1 Yêu cầu thiết kế và lựa chọn linh kiện 11

2.1.1 Yêu cầu của hệ thống 11

2.1.2 Lựa chọn cảm biến nhiệt độ 11

2.1.3 Lựa chọn thiết bị hiển thị và cảnh báo 13

2.2 Thiết kế kiến trúc phần cứng và sơ đồ nguyên lý 13

2.2.1 Sơ đồ khối chức năng và kết nối liên khối hệ thống 13

2.2.2 Bảng kết nối dây chi tiết giữa các linh kiện 14

2.3 Xây dựng thuật toán và phần mềm điều khiển 14
2.3.1 Thuật toán điều khiển tổng quát chương trình 14
2.3.2 Thuật toán giao tiếp và giải mã cảm biến DHT11 17
2.3.3 Xây dựng thư viện điều khiển màn hình LCD 1602 20
2.3.4 Thuật toán so sánh ngưỡng cảnh báo và điều khiển LED 21
2.4 Quy trình biên dịch và nạp firmware 22
2.4.1 Quy trình tự động hóa với Makefile 22
2.4.2 Quy trình nạp chương trình bằng phần mềm ProgISP 23

3 KẾT QUẢ THỰC NGHIỆM 25
3.1 Kết quả thực nghiệm trên phần cứng 25
3.1.1 Giai đoạn khởi động và màn hình chào mừng 25
3.1.2 Trạng thái đo lường bình thường ($T < 30°C$) 25
3.1.3 Trạng thái cảnh báo vượt ngưỡng quá nhiệt ($T \geq 30°C$) 26
3.1.4 Kiểm tra phản hồi khi mất cảm biến và xử lý lỗi hệ thống 27
3.2 Đánh giá kết quả và phạm vi kiểm chứng 28
3.2.1 Kết quả chức năng đã ghi nhận 28
3.2.2 Giới hạn của kết quả thực nghiệm 29

4 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 30
4.1 Kết luận 30
4.2 Hạn chế của đề tài 30
4.3 Hướng phát triển mở rộng 31

TÀI LIỆU THAM KHẢO 33

PHỤ LỤC: TOÀN BỘ MÃ NGUỒN CHƯƠNG TRÌNH 34

DANH MỤC HÌNH VẼ

1.1 Sơ đồ phân bố chân của vi điều khiển ATmega16 dạng đóng vỏ DIP-40 1

1.2 Mô hình 3D bố trí linh kiện trên Kit phát triển vi điều khiển AVR 4

2.1 Sơ đồ kết nối liên khối chi tiết của hệ thống đo nhiệt độ và cảnh báo . . 14

2.2 Sơ đồ nguyên lý kit AVR được nhóm vẽ lại từ tài liệu giảng viên cung cấp 15

2.3 Mô hình 3D PCB của nhóm trên Altium Designer 17

2.4 Lưu đồ thuật toán đo nhiệt độ, cảnh báo và xử lý lỗi 18

2.5 Giao diện ProgISP khi nạp firmware và cấu hình fuse `0xD9E4` 24

3.1 Hình ảnh thực tế hệ thống vận hành trong các kịch bản thử nghiệm . . . 27

DANH MỤC BẢNG

1.1 Thông số kỹ thuật chính của vi điều khiển ATmega16 2

1.2 Đặc tính kỹ thuật của cảm biến số DHT11 6

1.3 Bảng ánh xạ kết nối chân giữa giắc J7 trên Kit, màn hình LCD 1602 và ATmega16 9

2.1 Yêu cầu thiết kế của hệ thống 11

2.2 So sánh các phương án cảm biến nhiệt độ 12

2.3 Bảng tổng hợp kết nối tín hiệu phần cứng của hệ thống 16

2.4 Thời gian timeout của các vòng chờ trong `DHT11_Read` ở 8 MHz . . . 19

2.5 Dung lượng bộ nhớ của chương trình (`avr-size`, avr-gcc 16.1.0, `-Os`) 23

3.1 Bảng tổng hợp các kịch bản kiểm tra và kết quả đánh giá hệ thống . . 28

DANH MỤC TỪ VIẾT TẮT

Viết tắt	Ý nghĩa
ADC	Analog-to-Digital Converter – Bộ chuyển đổi tương tự – số
AVR	Họ vi điều khiển 8-bit của Atmel (nay thuộc Microchip)
DIP	Dual In-line Package – Kiểu đóng vỏ hai hàng chân cắm
EEPROM	Electrically Erasable Programmable Read-Only Memory
GPIO	General-Purpose Input/Output – Chân vào/ra đa dụng
I/O	Input/Output – Vào/ra
ISP	In-System Programming – Nạp chương trình trên mạch
JTAG	Joint Test Action Group – Chuẩn giao diện kiểm tra, gỡ lỗi (IEEE 1149.1)
LCD	Liquid Crystal Display – Màn hình tinh thể lỏng
LED	Light-Emitting Diode – Điốt phát quang
MCU	Microcontroller Unit – Vi điều khiển
NTC	Negative Temperature Coefficient – Hệ số nhiệt điện trở âm
PCB	Printed Circuit Board – Mạch in
PWM	Pulse Width Modulation – Điều chế độ rộng xung
RC	Resistor–Capacitor – Mạch dao động điện trở – tụ điện
RH	Relative Humidity – Độ ẩm tương đối
RISC	Reduced Instruction Set Computer – Kiến trúc tập lệnh rút gọn
SPI	Serial Peripheral Interface – Giao tiếp ngoại vi nối tiếp
SRAM	Static Random-Access Memory – Bộ nhớ truy cập ngẫu nhiên tĩnh
USART	Universal Synchronous/Asynchronous Receiver-Transmitter
USB	Universal Serial Bus

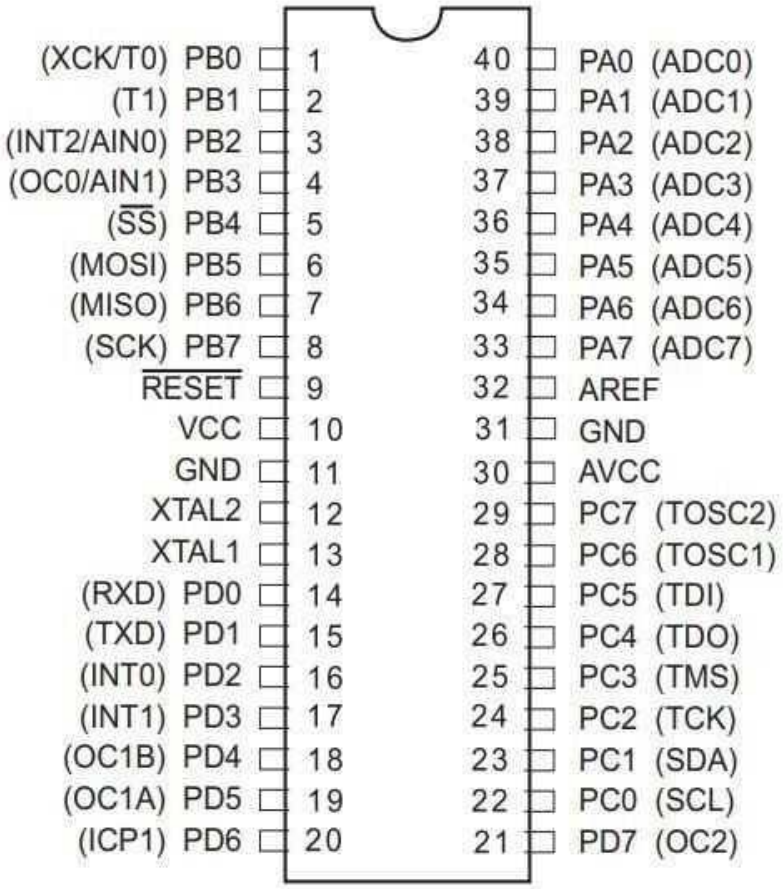
CHƯƠNG 1

GIỚI THIỆU TỔNG QUAN PHẦN CỨNG VÀ LINH KIỆN

1.1 Vi điều khiển ATmega16

1.1.1 Tổng quan kiến trúc và đặc tính kỹ thuật

ATmega16 là vi điều khiển 8-bit họ AVR của Atmel (nay thuộc Microchip Technology), sử dụng kiến trúc RISC với 131 lệnh, phần lớn thực thi trong một chu kỳ clock [1]. Thông lượng xử lý có thể đạt gần 1 MIPS trên mỗi MHz, phù hợp với yêu cầu đọc cảm biến và điều khiển hiển thị của mô hình.



Hình 1.1: Sơ đồ phân bố chân của vi điều khiển ATmega16 dạng đóng vỏ DIP-40

Các thông số kỹ thuật then chốt của vi điều khiển ATmega16 được tổng hợp trong Bảng 1.1:

Bảng 1.1: Thông số kỹ thuật chính của vi điều khiển ATmega16

Thông số	Giá trị định mức / Đặc tính kỹ thuật
Kiến trúc CPU	8-bit AVR RISC với 32 thanh ghi làm việc đa năng 8-bit
Bộ nhớ Flash chương trình	16 Kbytes (In-System Reprogrammable Flash, độ bền 10.000 lần ghi/xóa)
Bộ nhớ SRAM nội	1 Kbyte (1024 Bytes)
Bộ nhớ EEPROM nội	512 Bytes (Lưu trữ dữ liệu không mất khi mất điện, độ bền 100.000 lần)
Tần số xung nhịp tối đa	0 – 16 MHz (Điện áp 4.5V – 5.5V)
Số chân I/O khả trình	32 chân chia làm 4 cổng 8-bit (PORT A, PORT B, PORT C, PORT D)
Bộ định thời / Bộ đếm (Timer)	2 bộ Timer/Counter 8-bit và 1 bộ Timer/Counter 16-bit đa chức năng
Các kênh PWM	4 kênh điều chế độ rộng xung phần cứng
Bộ chuyển đổi ADC	8 kênh ADC 10-bit tích hợp sẵn trên PORT A
Giao tiếp truyền thông	USART, SPI, I2C/TWI
Giao diện gỡ lỗi phần cứng	Chuẩn JTAG (IEEE std. 1149.1) hỗ trợ On-Chip Debug
Điện áp hoạt động	4.5V – 5.5V đối với ATmega16 chuẩn (2.7V – 5.5V đối với ATmega16L)

1.1.2 Cấu trúc các cổng vào/ra và chức năng trong đồ án

Vi điều khiển ATmega16 trang bị 32 chân vào ra hai chiều, được phân bổ đều cho 4 cổng ngoại vi ký hiệu từ A đến D. Mỗi cổng được kiểm soát độc lập bởi 3 thanh ghi điều khiển địa chỉ bộ nhớ:

- Thanh ghi DDRx (hướng dữ liệu): Thiết lập cấu hình hướng cho từng chân (Bit = 0: Chân là đầu vào Input; Bit = 1: Chân là đầu ra Output).
- Thanh ghi PORTx (dữ liệu xuất): Khi chân được thiết lập là Output, việc ghi 0 hoặc 1 vào bit tương ứng sẽ xuất điện áp mức thấp (0V) hoặc mức cao (5V). Khi chân là Input, ghi bit 1 sẽ kích hoạt điện trở kéo lên nội, ghi bit 0 sẽ đưa chân về trạng thái trở kháng cao.
- Thanh ghi PINx (đọc dữ liệu vào): Đọc trạng thái logic điện áp thực tế tại các chân

cổng.

Trong đề tài, PA1 nối với chân dữ liệu của DHT11, PC4 – PC7 và PD5 – PD7 dùng cho LCD 1602, PD0 – PD4 điều khiển 5 LED cảnh báo; bảng kết nối đầy đủ ở Bảng 2.3.

1.1.3 Xung nhịp hệ thống và phân tích xung đột ngoại vi JTAG

Cấu hình nguồn xung nhịp 8.0 MHz bằng Fuse Bits

ATmega16 tích hợp sẵn mạch dao động RC nội đã hiệu chuẩn, có khả năng tạo ra các tần số xung nhịp cố định gồm 1.0 MHz, 2.0 MHz, 4.0 MHz và 8.0 MHz mà không đòi hỏi phải gắn thêm thạch anh dao động ngoài hay tụ lọc phụ trợ.

Sơ đồ kit có bố trí thạch anh ngoài `XT1` và hai tụ tải. Cấu hình phần mềm của đồ án chọn dao động RC nội 8.0 MHz, do đó không sử dụng khối thạch anh ngoài. Giá trị `F_CPU = 8000000UL` chỉ phục vụ tính toán trễ khi biên dịch; tần số phần cứng được chọn bằng Fuse Bits. Fuse được ghi ở bước nạp chương trình bằng ProgISP (mục 2.4.2, Hình 2.5) với cấu hình:

- Low Fuse Byte = `0xE4`: Thiết lập các bit `CKSEL3..0 = 0100` chọn mạch dao động RC nội 8.0 MHz, thời gian khởi động là 6 chu kỳ clock cộng trễ bổ sung điển hình 65 ms ở 5V (`SUT1..0 = 10`, Bảng 10 của datasheet).

Phân tích xung đột phần cứng JTAG trên PORT C

Mặc định khi xuất xưởng từ nhà sản xuất, bit cầu chì `JTAGEN` trên High Fuse Byte luôn được kích hoạt sẵn (`JTAGEN = 0`). Khi giao tiếp JTAG hoạt động, phần cứng vi điều khiển sẽ chiếm dụng cố định 4 chân trên cổng C:

- Chân PC2 = TCK (Test Clock)
- Chân PC3 = TMS (Test Mode Select)
- Chân PC4 = TDO (Test Data Output)
- Chân PC5 = TDI (Test Data Input)

Bus dữ liệu 4 bit của LCD 1602 nối vào PC4 – PC7. Khi JTAG còn bật, hai chân PC4 và PC5 do khối gỡ lỗi điều khiển và không còn hoạt động như GPIO, nên LCD không nhận được chuỗi lệnh khởi tạo. Triệu chứng thường gặp là dòng 1 hiện toàn ô đen, dòng 2 để trống.

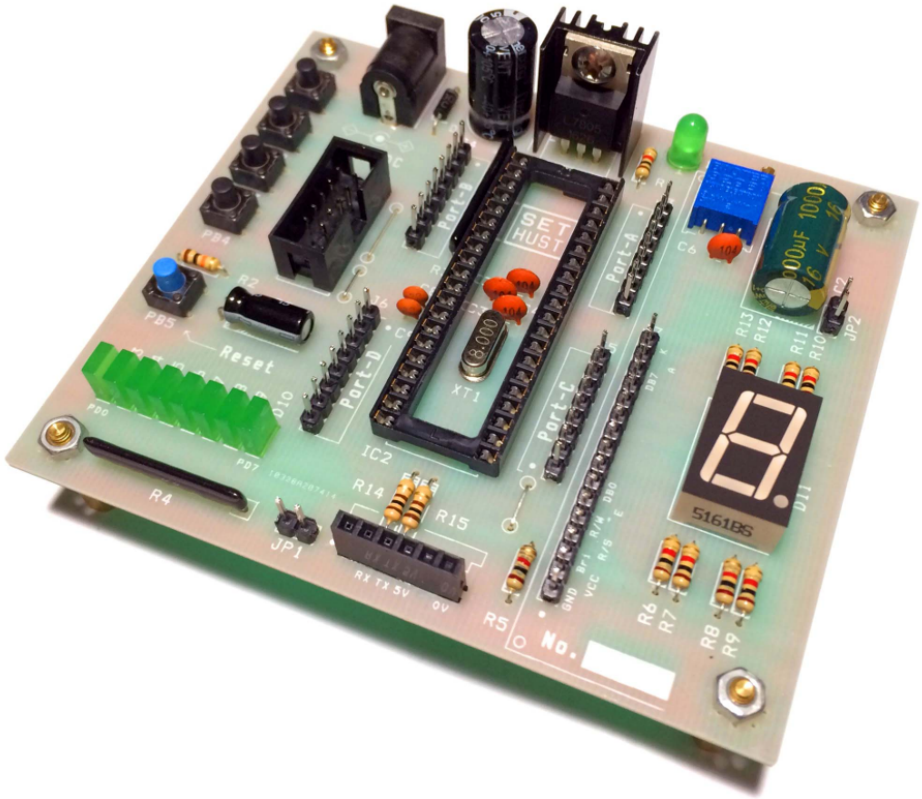
Để dùng PC4, PC5 cho LCD, đề tài tắt JTAG theo hai cách:

1. Tắt JTAG ở mức phần cứng: Cấu hình High Fuse Byte = `0xD9`. Tại giá trị này, bit 6 (`JTAGEN`) mang giá trị 1 (vô hiệu hóa JTAG), bit 5 (`SPIEN`) mang giá trị 0 (duy trì cổng nạp SPI để nạp code qua USBasp).
2. Tắt JTAG ở mức phần mềm: Ghi bit `JTD` trên thanh ghi `MCUCSR` hai lần liên tiếp trong vòng 4 chu kỳ máy ngay ở đầu hàm `main()`:

```
MCUCSR = (1 << JTD);
MCUCSR = (1 << JTD);
```

1.2 Mạch Kit thực hành vi điều khiển họ AVR

Kit AVR sử dụng trong đồ án được triển khai từ sơ đồ nguyên lý do giảng viên cung cấp [6]. Nhóm vẽ lại sơ đồ và thiết kế PCB trên Altium Designer; bố trí linh kiện được thể hiện ở Hình 1.2.



Hình 1.2: Mô hình 3D bố trí linh kiện trên Kit phát triển vi điều khiển AVR

Kit phát triển tích hợp đầy đủ các khối chức năng ngoại vi cơ bản:

1. Đế cắm vi điều khiển DIP-40: Cho phép lắp và thay thế vi điều khiển trong quá trình thực hành.
2. Khối nguồn nuôi và ổn áp 5V: Tích hợp IC ổn áp tuyến tính LM7805 hạ áp từ nguồn Adapter ngoài DC 9V–12V, đồng thời hỗ trợ nhận nguồn 5V trực tiếp từ cổng USB qua mạch nạp USBasp, kết hợp hệ thống tụ lọc nguồn và đèn LED chỉ thị nguồn VCC.
3. Giao diện nạp chương trình chuẩn ISP 10 chân (IDC-10): Kết nối trực tiếp với mạch nạp USBasp qua các đường tín hiệu MOSI (PB5), MISO (PB6), SCK (PB7), RESET và GND.
4. Hàng rào cắm mở rộng J7 cho LCD 1602: Chuẩn cắm hàng đơn 16 chân tương thích với các màn hình chuẩn HD44780, tích hợp sẵn biến trở vi chỉnh VR1 dùng để điều chỉnh độ tương phản.
5. Dàn LED đơn và phím bấm chức năng: Hệ thống 8 LED đơn nối về PORT D và ma trận nút bấm điều khiển nối về PORT B phục vụ các bài toán thực hành giao tiếp vào ra cơ bản. Đồ án dùng 5 LED trên PD0–PD4 để cảnh báo; 3 LED còn lại trên PD5–PD7 nằm chung đường với tín hiệu điều khiển LCD nên sẽ sáng theo mức logic của RW, RS, EN.

1.3 Cảm biến số nhiệt độ và độ ẩm DHT11

1.3.1 Cấu tạo và nguyên lý hoạt động

DHT11 là cảm biến số đo nhiệt độ và độ ẩm môi trường tích hợp vi xử lý 8-bit hiệu năng cao bên trong để số hóa và hiệu chuẩn dữ liệu trước khi truyền ra ngoài [2]. Cấu tạo cảm biến bao gồm hai thành phần cảm nhận vật lý:

- Cảm biến độ ẩm kiểu điện trở: Sử dụng một phần tử màng polymer nhạy ẩm có các điện cực xen kẽ. Khi độ ẩm môi trường biến thiên, các phân tử hơi nước được hấp thụ vào bề mặt làm phân ly các ion dẫn, dẫn đến biến đổi điện trở của màng. Vi điều khiển 8-bit tích hợp bên trong DHT11 sẽ đo đạc sự thay đổi điện trở này và tính toán quy đổi thành giá trị độ ẩm tương đối (%RH).
- Cảm biến nhiệt độ NTC: Là một điện trở nhiệt bán dẫn có hệ số nhiệt điện trở âm. Khi nhiệt độ môi trường tăng lên, điện trở của linh kiện giảm xuống theo một

hàm phi tuyến được định trước. Vi điều khiển chuyển đổi giá trị điện trở này thành nhiệt độ thực tế theo thang đo độ Celsius (°C).

Bảng 1.2: Đặc tính kỹ thuật của cảm biến số DHT11

Thông số kỹ thuật	Giá trị danh định
Điện áp hoạt động	3.0V – 5.5V DC (theo bản tài liệu tham chiếu)
Dải đo nhiệt độ	0°C – 50°C
Độ chính xác đo nhiệt độ	±2.0°C
Độ phân giải nhiệt độ	1.0°C (bản DHT11 trong tài liệu tham chiếu)
Dải đo độ ẩm	20% – 90% RH
Độ chính xác đo độ ẩm	±5.0% RH
Độ phân giải độ ẩm	1% RH
Khoảng cách giữa các lần đọc	Tối thiểu 1 giây theo bản tài liệu tham chiếu; chương trình chọn trễ 2 giây mỗi vòng lặp
Dòng điện tiêu thụ	0.5 mA – 2.5 mA khi đo; 100 μA – 150 μA khi chờ
Chuẩn giao tiếp	Aosong 1-Wire Single-Bus (Đường truyền đơn tuyến chuyên biệt)

Module sử dụng trong mô hình là loại DHT11 gắn trên PCB, đầu nối 3 chân, mã bán hàng `LK_02463`. Mã này không xác định phiên bản chip hoặc bản datasheet. Khi đấu nối cần dựa trên nhãn VCC, DATA (hoặc S) và GND trên module, không áp dụng trực tiếp số chân của cảm biến rời 4 chân. Bảng 1.2 dùng làm cơ sở tham chiếu cho DHT11. Chương trình hiển thị trực tiếp Byte 2 (T_{int}) và Byte 3 (T_{dec}) lên LCD, nên các giá trị 26.5°C và 31.5°C ghi nhận ở Chương 3 cho thấy module thực tế trả về byte thập phân khác 0, khác với mức phân giải 1°C của bản tài liệu tham chiếu.

1.3.2 Giao thức truyền thông đơn tuyến Aosong 1-Wire

Phân định chuẩn giao thức: Giao thức truyền thông một dây (Single-Bus) của DHT11 do nhà sản xuất Aosong thiết kế là giao thức điểm–điểm chuyên biệt, **không tương thích** với chuẩn công nghiệp Dallas/Maxim 1-Wire (vốn sử dụng mã định danh ROM 64-bit, khe thời gian do Master chủ động điều khiển và hỗ trợ kết nối đa điểm trên cùng một bus). Giao thức của DHT11 hoạt động theo nguyên tắc Master phát xung khởi động kích hoạt một lần, sau đó Slave (cảm biến) tự động phát liên tục chuỗi 40 bit dữ liệu theo các mốc thời gian quy ước.

Cảm biến DHT11 giao tiếp với vi điều khiển chỉ thông qua một chân dữ liệu DATA

duy nhất. Đường tín hiệu này đòi hỏi phải có một điện trở kéo lên nguồn 5V (giá trị từ 4.7 kΩ đến 10 kΩ) để duy trì mức cao khi đường truyền ở trạng thái rảnh rỗi.

Toàn bộ quy trình trao đổi dữ liệu một lần đọc (gồm 40 bit) tuân theo trình tự định thì thời gian nghiêm ngặt được mô tả dưới đây:

1. Tín hiệu khởi động

Vi điều khiển phát tín hiệu khởi động:

- MCU cấu hình chân PA1 thành Output và kéo đường bus xuống mức LOW trong thời gian tối thiểu 18 ms (trong chương trình chọn trễ an toàn là `_delay_ms(20)`).
- MCU sau đó giải phóng quyền điều khiển bus bằng cách chuyển chân PA1 sang chế độ Input và kích hoạt điện trở kéo lên nội, tạo khoảng trễ 30 μs để đường tín hiệu được kéo lên mức HIGH an toàn trước khi cảm biến bắt đầu phản hồi. Việc chuyển sang đầu vào cho phép cảm biến điều khiển đường DATA mà không bị MCU chủ động giữ mức cao.

2. Tín hiệu phản hồi từ cảm biến

Khi nhận diện được xung khởi động hợp lệ, DHT11 sẽ kéo đường bus xuống mức LOW trong khoảng 80 μs, tiếp theo kéo lên mức HIGH trong 80 μs để thông báo đã sẵn sàng truyền gói dữ liệu.

3. Thu nhận và giải mã 40 bit dữ liệu

Sau khi gửi tín hiệu phản hồi, DHT11 bắt đầu tuần tự phát ra 40 bit nhị phân (chia đều thành 5 byte). Mỗi bit dữ liệu luôn được bắt đầu bằng một xung mức LOW kéo dài cố định khoảng 50 μs. Giá trị của bit nhị phân được mã hóa dựa trên độ rộng thời gian tồn tại của xung mức HIGH tiếp theo đó:

- Bit ’0’: Xung mức HIGH tồn tại trong khoảng từ 26 μs đến 28 μs.
- Bit ’1’: Xung mức HIGH tồn tại kéo dài trong khoảng 70 μs.

Cách vi điều khiển phân biệt hai độ rộng xung này được trình bày ở mục 2.3.2.

4. Cấu trúc gói tin và cơ chế kiểm tra lỗi

Gói dữ liệu hoàn chỉnh gồm 5 byte (40 bit) truyền đi theo thứ tự:

$$\begin{aligned}\text{Gói dữ liệu} = \{&\text{Byte 0: } RH_{int}, \quad \text{Byte 1: } RH_{dec}, \\ &\text{Byte 2: } T_{int}, \quad \text{Byte 3: } T_{dec}, \quad \text{Byte 4: } Checksum\}\end{aligned} \tag{1.1}$$

Byte cuối cùng là byte kiểm tra lỗi Checksum được tạo bởi tổng đại số của 4 byte dữ liệu trước đó (lấy 8 bit thấp):

$$Checksum_{valid} = (RH_{int} + RH_{dec} + T_{int} + T_{dec}) \ \& \ 0\text{xFF} \tag{1.2}$$

Nếu giá trị tính toán trùng khớp với Byte 4 nhận được từ cảm biến, gói dữ liệu được xác nhận là hợp lệ. Thuật toán Checksum 8-bit giúp phát hiện hiệu quả các trường hợp méo dạng xung hoặc sai lệch bit đơn lẻ trên đường truyền; tuy nhiên, phương pháp tổng kiểm tra cộng 8-bit không có khả năng sửa lỗi và không thể loại trừ hoàn toàn các lỗi đảo bit bù trừ đa điểm.

1.4 Màn hình hiển thị tinh thể lỏng LCD 1602

Màn hình LCD 1602 là mô-đun hiển thị ký tự gồm 2 hàng, mỗi hàng có khả năng hiển thị tối đa 16 ký tự ma trận điểm 5 × 8 pixels, sử dụng vi mạch điều khiển tiêu chuẩn công nghiệp Hitachi HD44780 [5].

Bảng 1.3: Bảng ánh xạ kết nối chân giữa giắc J7 trên Kit, màn hình LCD 1602 và ATmega16

Chân J7	Chân LCD	Tên chân	**Chân MCU**	Chức năng kỹ thuật
1	1	V_{SS}	**GND**	Cực âm nguồn nuôi (0V)
2	2	V_{DD}	**VCC**	Cực dương nguồn nuôi (+5V)
3	3	V_0	**VR1**	Điện áp điều chỉnh độ tương phản hiển thị màn hình
4	4	RS	**PD6**	Chọn thanh ghi (0: Thanh ghi Lệnh; 1: Thanh ghi Dữ liệu)
5	5	RW	**PD5**	Chọn chiều Đọc/Ghi (0: Ghi dữ liệu vào LCD; 1: Đọc từ LCD)
6	6	E	**PD7**	Chốt xung kích hoạt (Enable Strobe, tích cực sườn âm)
7 – 10	7 – 10	D0 – D3	**PC0 – PC3**	4 bit dữ liệu thấp (Không dùng ở chế độ 4-bit bus)
11	11	D4	**PC4**	Bit dữ liệu D4
12	12	D5	**PC5**	Bit dữ liệu D5
13	13	D6	**PC6**	Bit dữ liệu D6
14	14	D7	**PC7**	Bit dữ liệu D7
15	15	A (LED+)	**VCC**	Cực anot đèn nền
16	16	K (LED-)	**GND**	Cực katot đèn nền

Để tối ưu hóa số lượng chân vi điều khiển, đồ án áp dụng chế độ truyền thông 4-bit bus:

- Vi điều khiển chỉ cần dùng 4 chân GPIO (PC4 – PC7) nối tới các chân dữ liệu D4 – D7 của LCD, tiết kiệm được 4 chân so với chế độ 8-bit truyền thống.
- Mỗi byte dữ liệu hoặc mã lệnh 8-bit sẽ được tách làm hai nửa byte: 4 bit cao được truyền trước kèm theo một xung chốt Enable trên chân PD7, tiếp theo 4 bit thấp được dịch sang và truyền kèm theo một xung chốt Enable thứ hai.

1.5 Khối cảnh báo và chỉ thị LED

Dàn 5 đèn LED cảnh báo trên Kit thí nghiệm được nối trực tiếp vào các chân từ PD0 đến PD4 của cổng D. Mạch được thiết kế theo cấu hình tích cực mức thấp (Active

Low):

- Cực Anot của các đèn LED được cấp nguồn dương VCC qua mạng điện trở hạn dòng R4 và jumper JP1. Trên sơ đồ kit, R4 được ghi giá trị 10 kΩ.
- Cực Katot của các đèn LED nối về các chân vi điều khiển PD0 – PD4.
- Khi vi điều khiển xuất mức 0 (mức thấp): Dòng điện chạy từ VCC qua LED xuống chân vi điều khiển khép mạch, LED bật sáng (báo động vượt ngưỡng).
- Khi vi điều khiển xuất mức 1 (mức cao): Không có chênh lệch điện áp qua diode, LED tắt (trạng thái không cảnh báo quá nhiệt).

CHƯƠNG 2

THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG

2.1 Yêu cầu thiết kế và lựa chọn linh kiện

2.1.1 Yêu cầu của hệ thống

Từ tên đề tài và điều kiện thực hiện trên kit AVR của học phần, nhóm đặt ra các yêu cầu thiết kế trong Bảng 2.1. Vi điều khiển ATmega16 có sẵn trên kit nên không thuộc phạm vi lựa chọn; các mục tiếp theo trình bày việc chọn cảm biến và thiết bị hiển thị, cảnh báo.

Bảng 2.1: Yêu cầu thiết kế của hệ thống

Nhóm yêu cầu	Chỉ tiêu	Cách đáp ứng
Đo nhiệt độ	Nhiệt độ môi trường trong phòng, dải tối thiểu 0–50 °C; cập nhật khoảng 2 giây một lần	DHT11 (mục 2.1.2); chu kỳ đọc (mục 2.3.4)
Hiển thị	Hiện giá trị đo, ngưỡng và thông báo lỗi bằng chữ ngay trên mô hình	LCD 1602 (mục 2.1.3)
Cảnh báo	Báo khi nhiệt độ đạt hoặc vượt 30 °C bằng tín hiệu nhìn thấy được; tự trở về khi nhiệt độ giảm	5 LED trên PD0–PD4 (mục 2.3.4)
Độ tin cậy	Phát hiện mất tín hiệu và gói tin sai; vi điều khiển không bị treo khi cảm biến hỏng	Timeout và checksum (mục 2.3.2)
Phần cứng	Dùng kit ATmega16 có sẵn, nguồn 5 V, nạp qua USBasp; hạn chế linh kiện ngoài kit	Kết nối ở Bảng 2.3

2.1.2 Lựa chọn cảm biến nhiệt độ

Bảng 2.2 so sánh ba cảm biến nhiệt độ phổ biến khi làm việc với vi điều khiển AVR [2, 3, 4].

Bảng 2.2: So sánh các phương án cảm biến nhiệt độ

Tiêu chí	LM35	DS18B20	DHT11
Tín hiệu ra	Điện áp tương tự, 10 mV/°C	Số, bus 1-Wire (Dallas)	Số, Single-Bus (Aosong)
Dải đo nhiệt độ	−55 đến 150 °C	−55 đến 125 °C	0 đến 50 °C
Độ chính xác	0,5 °C tại 25 °C	±0,5 °C trong khoảng −10 đến 85 °C	±2 °C
Độ phân giải	Phụ thuộc ADC: khoảng 0,49 °C với ADC 10 bit, tham chiếu 5 V	0,5 đến 0,0625 °C (9–12 bit)	1 °C theo tài liệu tham chiếu
Nguồn cấp	4–30 V	3,0–5,5 V	3,0–5,5 V
Thời gian đọc	Một lần chuyển đổi ADC	Tối đa 750 ms ở 12 bit	Khoảng 25 ms; hai lần đọc cách nhau tối thiểu 1 s
Đo độ ẩm	Không	Không	Có (20–90 % RH)
Kết nối với MCU	Một kênh ADC; dây tín hiệu tương tự dễ nhiễm nhiễu	Một chân GPIO, điện trở kéo lên	Một chân GPIO, điện trở kéo lên

LM35 và DS18B20 chính xác hơn DHT11. Nhóm vẫn chọn DHT11 vì các lý do sau:

- Dải 0–50 °C và sai số ±2 °C đáp ứng yêu cầu cảnh báo ngưỡng 30 °C trong phòng ở Bảng 2.1.
- Tín hiệu số chỉ cần một chân GPIO (PA1); không chiếm kênh ADC và không phụ thuộc điện áp tham chiếu hay nhiễu trên dây như LM35.
- Module 3 chân dễ đấu nối ngoài kit; cảm biến đo thêm độ ẩm, cho phép mở rộng chức năng mà không đổi phần cứng.
- Giao thức buộc chương trình tự tạo và lấy mẫu xung theo thời gian, phù hợp với nội dung lập trình thanh ghi của học phần.

Đổi lại, sai số của DHT11 lớn hơn hai phương án còn lại. Nếu ứng dụng cần độ chính xác cao hơn, DS18B20 có thể thay trên cùng chân PA1, với thay đổi chủ yếu ở phần mềm giao tiếp.

2.1.3 Lựa chọn thiết bị hiển thị và cảnh báo

Kit có sẵn ba thiết bị đầu ra dùng được: LED 7 thanh một chữ số D11, dãy 8 LED đơn trên PORT D và giắc J7 dành cho LCD 1602 (Hình 2.2). LED 7 thanh một chữ số không đủ để hiện giá trị như 26.5 hay các thông báo lỗi, đồng thời dùng chung PC0–PC7 với giắc LCD. Nhóm chọn LCD 1602 (2 dòng × 16 ký tự, bộ điều khiển HD44780) vì đủ chỗ cho giá trị đo, ngưỡng và thông báo chẩn đoán bằng chữ; LCD chạy ở chế độ 4 bit để chỉ chiếm PC4–PC7 và PD5–PD7.

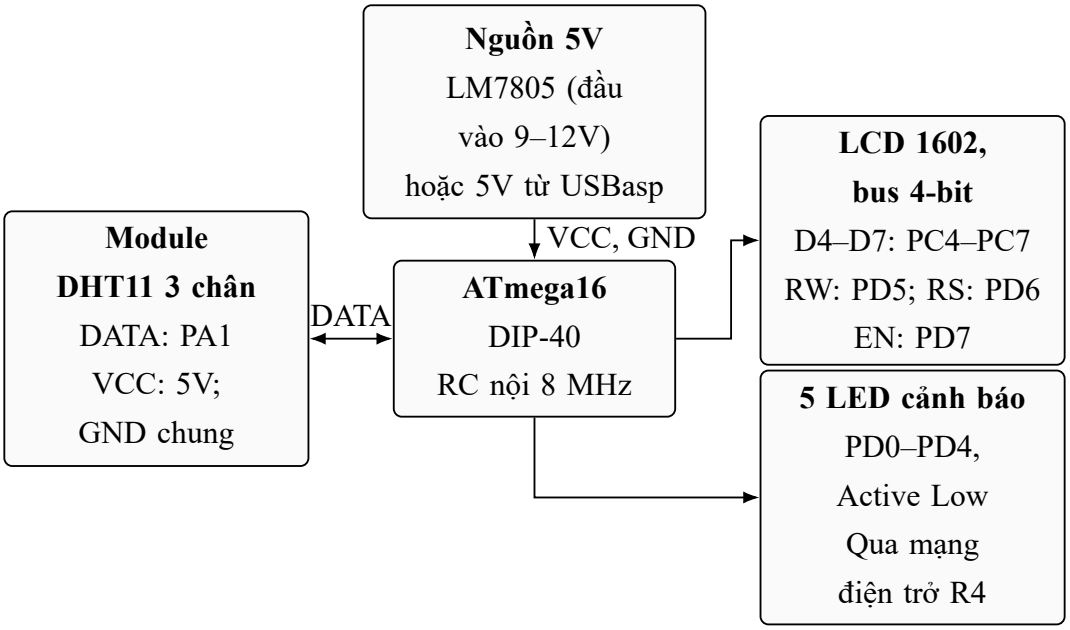
Khối cảnh báo dùng 5 LED đơn trên PD0–PD4: các LED có sẵn trên kit, không cần mạch ngoài và dễ quan sát. Còi báo không có trên kit nên được đưa vào hướng phát triển (mục 4.3).

2.2 Thiết kế kiến trúc phần cứng và sơ đồ nguyên lý

2.2.1 Sơ đồ khối chức năng và kết nối liên khối hệ thống

Nhóm vẽ lại sơ đồ giảng viên cung cấp (Hình 2.2) [6] và thiết kế PCB trên Altium Designer (Hình 2.3). DHT11 nối ngoài kit theo Hình 2.1. Hệ thống gồm 5 khối chức năng:

1. **Khối nguồn nuôi 5V**: Cung cấp điện áp +5V ổn định cho toàn mạch qua IC LM7805 hoặc qua cổng USB máy tính.
2. **Khối MCU trung tâm**: ATmega16 (DIP-40, dao động nội 8.0 MHz) đảm nhiệm điều phối toàn bộ chu trình hoạt động.
3. **Khối cảm biến**: Cảm biến DHT11 thu thập giá trị nhiệt độ và độ ẩm, giao tiếp đơn tuyến với MCU qua chân PA1.
4. **Khối hiển thị**: Màn hình LCD 1602 giao tiếp 4-bit qua các chân dữ liệu PC4–PC7 và chân điều khiển PD5–PD7.
5. **Khối cảnh báo**: Dàn 5 LED đơn nối với chân PD0–PD4, tích cực mức thấp qua mạng điện trở R4 trên kit.



Hình 2.1: Sơ đồ kết nối liên khối chi tiết của hệ thống đo nhiệt độ và cảnh báo

2.2.2 Bảng kết nối dây chi tiết giữa các linh kiện

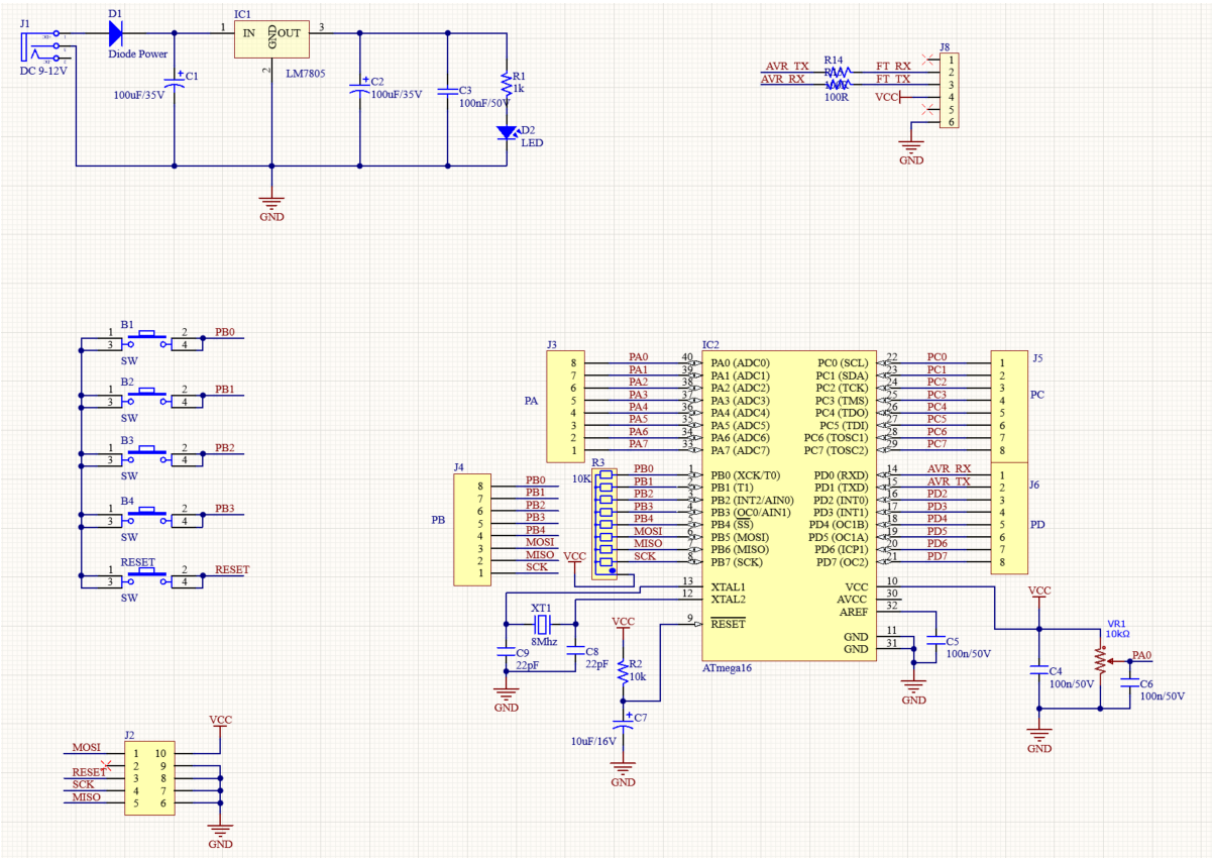
Bảng 2.3 liệt kê kết nối giữa từng khối chức năng và chân của ATmega16; số chân tính theo vỏ DIP-40 (Hình 1.1). Tên tín hiệu sau dấu "/" là chức năng thay thế của chân, không được dùng trong đồ án.

2.3 Xây dựng thuật toán và phần mềm điều khiển

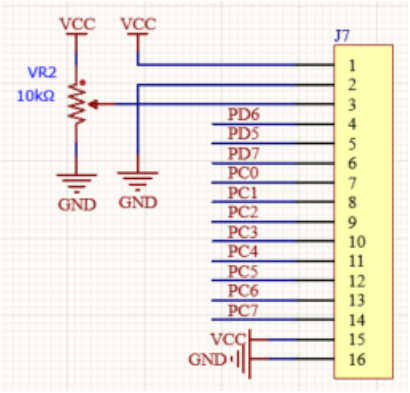
2.3.1 Thuật toán điều khiển tổng quát chương trình

Chương trình vận hành tuần tự theo các bước dưới đây. Lưu đồ ở Hình 2.4 thể hiện cùng các nhánh xử lý; mã nguồn đầy đủ nằm ở Phụ lục A.

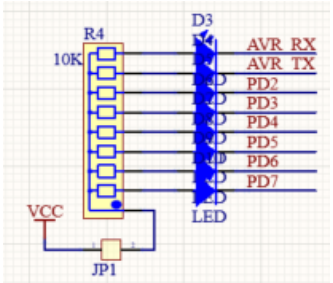
1. **Khởi tạo hệ thống.** Ghi hai lần bit JTD trong `MCUCSR` để tắt JTAG và giải phóng PC4, PC5. Khởi tạo LCD bằng `LCD4_INIT`; đặt PD0–PD4 làm đầu ra (`DDRD |= 0x1F`) và xuất mức cao để tắt LED (`PORTD |= 0x1F`). LCD hiện `DA2: DO NHIET DO` và `DHT11 SENSOR` khoảng 1,5 giây, sau đó được xóa.
2. **Đọc cảm biến.** Gọi `DHT11_Read` để nhận dữ liệu nhiệt độ cùng mã trả về `status`. Chương trình chỉ so sánh ngưỡng và hiển thị nhiệt độ khi `status == 0`.
3. **Xử lý dữ liệu hợp lệ.** Dòng 1 LCD hiển thị nhiệt độ theo định dạng `Temp: XX.Y °C`. Dòng 2 hiển thị ngưỡng; các khoảng trắng cuối chuỗi được dùng để ghi đè nội dung cũ trong 16 vị trí:



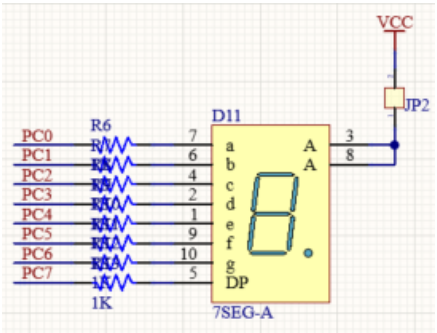
(a) Khối nguồn, vi điều khiển ATmega16, nút bấm, cổng nạp ISP (J2) và các giắc mở rộng cổng



(b) Giắc J7 nối LCD 1602



(c) Dãy LED đơn



(d) LED 7 thanh

Hình 2.2: Sơ đồ nguyên lý kit AVR được nhóm vẽ lại từ tài liệu giảng viên cung cấp

Ghi chú: Các khối (a)–(d) nối với nhau qua nhãn mạng (PA0–PD7, VCC, GND). Trong đồ án, LCD dùng PC4–PC7 và PD5–PD7; 5 LED cảnh báo dùng PD0–PD4. Khối thạch anh XT1 có trên sơ đồ nhưng chương trình chọn dao động RC nội 8 MHz. Cảm biến DHT11 là module nối ngoài, không nằm trong sơ đồ kit này.

Bảng 2.3: Bảng tổng hợp kết nối tín hiệu phần cứng của hệ thống

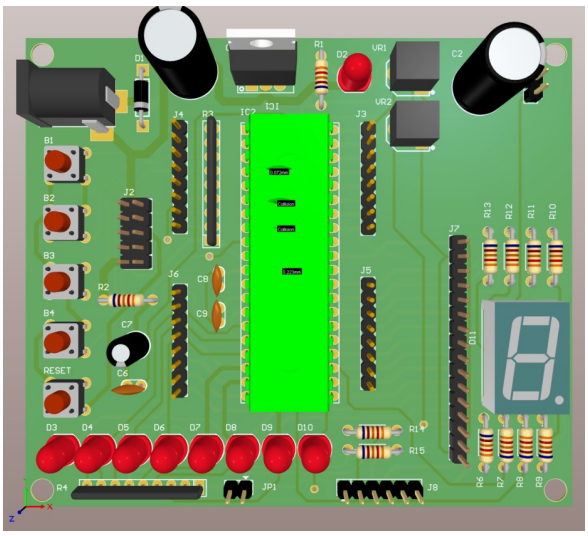
Khối chức năng	Chân linh kiện	Chân MCU	**Tên tín hiệu MCU**	Chức năng kỹ thuật
Cảm biến DHT11	DATA / S	Chân 39	**PA1 / ADC1**	Giao tiếp Single-Bus hai chiều
	VCC	–	**+5V của kit**	Nguồn nuôi +5V DC
	GND	–	**GND của kit**	Nối đất chung
Màn hình LCD	RS (Chân 4)	Chân 20	**PD6 / ICP1**	Chọn thanh ghi (0: lệnh, 1: dữ liệu)
	RW (Chân 5)	Chân 19	**PD5 / OC1A**	Chọn đọc/ghi (0: ghi)
	EN (Chân 6)	Chân 21	**PD7 / OC2**	Xung kích hoạt chốt dữ liệu
	D4 (Chân 11)	Chân 26	**PC4 / TDO**	Đường truyền dữ liệu bit 4
	D5 (Chân 12)	Chân 27	**PC5 / TDI**	Đường truyền dữ liệu bit 5
	D6 (Chân 13)	Chân 28	**PC6 / TOSC1**	Đường truyền dữ liệu bit 6
	D7 (Chân 14)	Chân 29	**PC7 / TOSC2**	Đường truyền dữ liệu bit 7
Dàn LED cảnh báo	Katot LED 1	Chân 14	**PD0 / RXD**	Cảnh báo, sáng khi mức 0
	Katot LED 2	Chân 15	**PD1 / TXD**	Cảnh báo, sáng khi mức 0
	Katot LED 3	Chân 16	**PD2 / INT0**	Cảnh báo, sáng khi mức 0
	Katot LED 4	Chân 17	**PD3 / INT1**	Cảnh báo, sáng khi mức 0
	Katot LED 5	Chân 18	**PD4 / OC1B**	Cảnh báo, sáng khi mức 0

- Nếu `temp_int >= 30`: hiện `HIGH Th: 30°C *!`; dùng `PORTD &= 0xE0` kéo PD0–PD4 xuống mức thấp, bật 5 LED.
- Nếu `temp_int < 30`: hiện `HIGH Th: 30°C`, xóa dấu `*!`; dùng `PORTD |= 0x1F` tắt 5 LED.

4. **Xử lý lỗi.** Không dùng lại dữ liệu cũ để so sánh ngưỡng. Cả hai nhánh lỗi đều tắt 5 LED bằng `PORTD |= 0x1F`:
   - `status == -6`: hiện `DHT11: Sum Error` và `Checksum Invalid` trên hai dòng LCD.
   - `status` từ -1 đến -5: hiện `DHT11: No Signal` và `Check Pin PA1`, tương ứng với timeout khi chờ tín hiệu.

   Việc tắt LED khi lỗi tránh cảnh báo dựa trên dữ liệu cũ, nhưng cũng làm hệ thống không cảnh báo được nếu môi trường đang quá nhiệt; hạn chế này được nêu ở mục 4.2.

5. **Lặp lại.** Thực hiện `_delay_ms(2000)`, rồi quay lại bước đọc cảm biến. Thời gian giữa hai lượt đọc còn bao gồm giao tiếp và cập nhật LCD, như phân tích ở mục 2.3.4.



Hình 2.3: Mô hình 3D PCB của nhóm trên Altium Designer

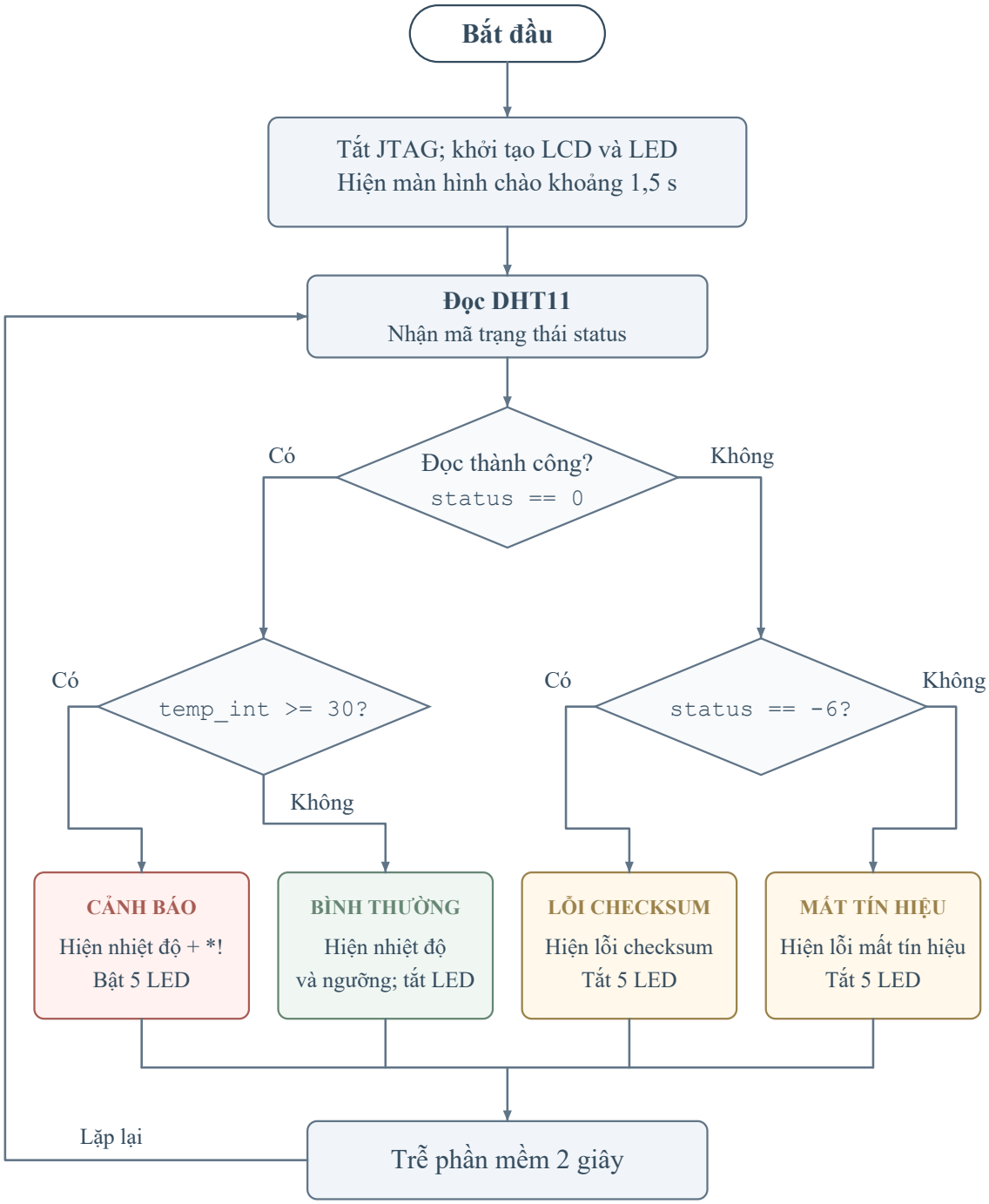
2.3.2 Thuật toán giao tiếp và giải mã cảm biến DHT11

Hàm `DHT11_Read(uint8_t *temp_int, uint8_t *temp_dec, uint8_t *humi_int)` được viết trực tiếp bằng thao tác thanh ghi để tạo và lấy mẫu xung theo các mốc thời gian của giao thức, kèm cơ chế timeout chống treo vi điều khiển.

Cơ chế chống treo bằng timeout

Nếu chân DATA bị hở mạch hoặc cảm biến hỏng, các vòng chờ sườn xung như `while(DHT_PIN_R & (1 << DHT_BIT))` sẽ lặp vô hạn và vi điều khiển bị treo. Vì vậy mỗi vòng chờ được gắn một biến đếm giảm dần `timeout = 10000`; khi biến đếm về 0 mà trạng thái chân chưa đổi, hàm thoát và trả về mã lỗi âm ứng với vòng chờ đó.

Thời gian timeout được xác định từ mã máy do `avr-gcc` sinh ra với tùy chọn `-Os` (xem bằng `avr-objdump -d`). Mỗi lần lặp của bốn vòng chờ đầu gồm lệnh kiểm tra bit `sbic/sbis` (1 chu kỳ), `rjmp` (2), `sbiw` (2) và `brne` (2), tổng cộng 7 chu kỳ. Ở vòng chờ thứ năm, lệnh `sbis` nhảy qua `rjmp` (2 chu kỳ) nên mỗi lần lặp tốn 6 chu kỳ. Kết quả ở 8 MHz được tổng hợp trong Bảng 2.4.



Hình 2.4: Lưu đồ thuật toán đo nhiệt độ, cảnh báo và xử lý lỗi

Bảng 2.4: Thời gian timeout của các vòng chờ trong `DHT11_Read` ở 8 MHz

Mã lỗi	Vòng chờ	Chu kỳ/lần lặp	Timeout
−1	DHT11 kéo đường DATA xuống mức thấp sau xung khởi động	7	8,75 ms
−2	DHT11 kéo đường DATA lên mức cao (tín hiệu phản hồi)	7	8,75 ms
−3	DHT11 kéo đường DATA xuống mức thấp trước bit đầu tiên	7	8,75 ms
−4	Kết thúc mức thấp 50 μs ở đầu mỗi bit	7	8,75 ms
−5	Kết thúc mức cao của bit '1'	6	7,5 ms

Khi rút dây DATA, điện trở kéo lên giữ PA1 ở mức cao, vòng chờ đầu tiên hết hạn và hàm trả về −1 sau khoảng 20 ms (xung khởi động) + 8,75 ms ≈ 29 ms, không đáng kể so với chu kỳ 2 giây. Các giá trị trên tỷ lệ nghịch với tần số thực của dao động RC nội. Cũng từ mã máy, `_delay_us(38)` được biên dịch thành vòng trễ 304 chu kỳ, đúng 38 μs ở 8 MHz.

Giải mã bit dữ liệu

Bit '0' có mức cao kéo dài 26–28 μs, bit '1' khoảng 70 μs (mục 1.3.2). Thời điểm lấy mẫu 38 μs nằm giữa hai giá trị này: nếu PA1 còn ở mức cao thì bit là '1', nếu đã về mức thấp thì bit là '0'. Sau xung khởi động và tín hiệu phản hồi, vòng lặp lồng nhau quét 5 byte (40 bit); mỗi bit được giải mã theo các bước:

1. Đợi kết thúc xung mức LOW 50 μs của bit đang truyền.
2. Đưa CPU vào trạng thái trễ 38 μs (`_delay_us(38)`).
3. Đọc thanh ghi `PINA`: nếu bit PA1 đang ở mức 1, ta ghi nhận đây là bit '1' bằng phép dịch bit:

```
data[i] |= (1 << j);
```

Tiếp tục chờ xung mức HIGH kết thúc trước khi đón nhận bit tiếp theo.

Kiểm tra checksum và mã trả về

Cuối hàm `DHT11_Read`, tổng của 4 byte đầu được so sánh với byte thứ 5 (byte checksum):

```
if ((uint8_t)(data[0] + data[1] + data[2] + data[3]) != data[4]) {
    return -6; // Lỗi sai Checksum
}
```

Nếu khớp, các con trỏ nhận giá trị tương ứng và hàm trả về 0. Như vậy hàm có ba nhóm mã trả về: 0 khi đọc thành công; −1 đến −5 khi hết thời gian ở một trong năm vòng chờ của Bảng 2.4; −6 khi sai checksum. Cách `main()` xử lý từng nhóm mã được trình bày ở bước 4 của mục 2.3.1.

2.3.3 Xây dựng thư viện điều khiển màn hình LCD 1602

Thư viện `hunget_lcd.h` được xây dựng với sự hỗ trợ của công cụ sinh mã và cấu hình cho cách nối chân của kit. Các hàm dưới đây trình bày nguyên lý truyền dữ liệu, tạo xung Enable và giữ nguyên các bit không dùng cho LCD; việc dùng công cụ hỗ trợ không thay thế bước kiểm tra trên phần cứng.

Hàm tạo xung chốt dữ liệu (`LCD_Enable`)

Theo chuẩn Hitachi HD44780, dữ liệu tại bus 4-bit chỉ được bộ điều khiển LCD ghi nhận tại thời điểm sườn xuống của chân Enable (chân PD7). Do vi điều khiển chạy ở tần số 8.0 MHz (mỗi chu kỳ clock dài 0.125 μs), hàm tạo xung cần bảo đảm thời gian thiết lập và thời gian giữ dữ liệu:

```
static inline void LCD_Enable(void) {
    LCD_CTRL_PORT |= (1 << LCD_EN);
    _delay_us(5);
    LCD_CTRL_PORT &= ~(1 << LCD_EN);
    _delay_us(100);
}
```

Hàm truyền 4 bit dữ liệu (`LCD_Send4Bit`)

Do cổng PORT C chỉ dùng 4 chân cao (PC4 – PC7) cho LCD, việc ghi dữ liệu ra LCD không được làm thay đổi trạng thái của 4 chân thấp (PC0 – PC3). Do đó chương trình dùng phép AND/OR với mặt nạ bit:

```
static inline void LCD_Send4Bit(unsigned char Data) {
    unsigned char current_port_c = LCD_DATA_PORT & 0x0F;
    LCD_DATA_PORT = current_port_c | (Data & 0xF0);
```

```
    LCD_Enable();
}
```

Trình tự khởi tạo chế độ 4-bit (LCD4_INIT)

Khi mới cấp điện, màn hình LCD có thể chưa ở trạng thái giao tiếp mong muốn. Trình tự khởi tạo phần mềm bằng chuỗi lệnh gửi 3 lần mã `0x30` cách nhau các khoảng trễ hợp lý giúp đưa vi mạch HD44780 về trạng thái reset đồng bộ, sau đó chuyển sang chế độ 4-bit bus (`0x20`) và cấu hình hiển thị 2 dòng ma trận 5 × 8 (`0x28`).

2.3.4 Thuật toán so sánh ngưỡng cảnh báo và điều khiển LED

Ngưỡng nhiệt độ cảnh báo của hệ thống được cố định tại giá trị 30°C (`HIGH Th: 30°C`). Đây là mốc kiểm thử để quan sát sự chuyển trạng thái của mô hình trong điều kiện phòng và gia nhiệt nhẹ, không phải ngưỡng an toàn được suy ra từ yêu cầu của một ứng dụng cụ thể. Với dữ liệu nhiệt độ không âm theo định dạng đang dùng, phép so sánh `temp_int >= 30` cũng kích hoạt cảnh báo tại đúng 30°C.

Hệ thống chưa có vùng trễ (hysteresis): cùng một ngưỡng được dùng để bật và tắt cảnh báo. Nếu số đo dao động quanh 30°C, LED có thể đổi trạng thái qua các lần đọc. Đây là hạn chế cần khảo sát ở các điểm sát ngưỡng.

Khi điều khiển dàn LED trên PORT D, thách thức kỹ thuật đặt ra là các chân PD5, PD6, PD7 đang đảm nhiệm điều khiển các chân tín hiệu RW, RS, EN của màn hình LCD. Nếu ghi trực tiếp một giá trị vào thanh ghi `PORTD`, các tín hiệu điều khiển LCD sẽ bị ghi đè và LCD có thể nhận sai lệnh hoặc dữ liệu.

Để giải quyết bài toán dùng chung cổng ngoại vi này, chương trình dùng mặt nạ bit:

- Khi nhiệt độ vượt ngưỡng (*temp_int* ≥ 30°C): Cần đưa 5 bit thấp (PD0 – PD4) về mức 0 (bật sáng LED), nhưng giữ nguyên giá trị của 3 bit cao (PD5 – PD7). Phép toán `AND` với mặt nạ `0xE0` (nhị phân `1110 0000`) được sử dụng:

$$\text{PORTD} = \text{PORTD} \,\&\, 0\text{xE0} \tag{2.1}$$

- Khi giá trị nhiệt độ dưới ngưỡng (*temp_int* < 30°C): Cần đưa 5 bit thấp (PD0 – PD4) lên mức 1 (tắt LED), đồng thời bảo toàn 3 bit cao (PD5 – PD7). Phép toán `OR` với mặt nạ `0x1F` (nhị phân `0001 1111`) được sử dụng:

$$\text{PORTD} = \text{PORTD} \mid 0\text{x1F} \tag{2.2}$$

Nhờ kỹ thuật che mặt nạ bit này, khối cảnh báo LED và khối hiển thị LCD cập nhật các nhóm chân riêng biệt trên cùng một cổng PORT D. Chuỗi cảnh báo hiển thị trên dòng 2 của màn hình LCD có định dạng `"HIGH Th: 30°C *!"` với đúng 16 ký tự, vừa đủ 16 vị trí hiển thị của một dòng LCD 1602. Sau mỗi vòng lặp, chương trình thực hiện `_delay_ms(2000)`. Chu kỳ giữa hai lần bắt đầu đọc liên tiếp còn bao gồm thời gian giao tiếp DHT11 và cập nhật LCD:

$$T_{\text{chu kỳ}} \approx 2.0\,\text{s} + T_{\text{đọc}} + T_{\text{LCD}} + T_{\text{xử lý}}. \tag{2.3}$$

Do đó chu kỳ thực tế lớn hơn 2 giây ở tần số danh định, không phải bộ định thời lấy mẫu chính xác 2 giây. Các khoảng trễ phụ thuộc tần số RC nội thực tế và cấu hình `F_CPU` [7].

2.4 Quy trình biên dịch và nạp firmware

2.4.1 Quy trình tự động hóa với Makefile

Mã nguồn dự án được tổ chức và biên dịch tự động hóa thông qua công cụ GNU Make và bộ biên dịch `avr-gcc` [7]. Tập tin `Makefile` thiết lập các quy tắc dịch mã:

- Thiết lập cờ biên dịch: `-Wall -Os -mmcu=atmega16 -DF_CPU=8000000UL` (tối ưu mã máy `-Os`, tần số 8 MHz).
- Quy tắc tạo file `tempsen.elf` từ file nguồn `tempsen.c` và file tiêu đề `hunget_lcd.h`.
- Quy tắc chuyển đổi định dạng `tempsen.hex`: Sử dụng công cụ `avr-objcopy -O ihex -R .eeprom tempsen.elf tempsen.hex` để tạo file mã máy tiêu chuẩn Intel HEX sẵn sàng nạp vào bộ nhớ Flash ROM.
- Lệnh `avr-size tempsen.elf` báo dung lượng bộ nhớ mà chương trình sử dụng; kết quả ở Bảng 2.5.

Cấu hình nạp fuse trong Makefile dùng `LFUSE = 0xE4` và `HFUSE = 0xD9`, thống nhất với nguồn clock RC nội 8 MHz trong báo cáo. Lệnh `make fuse` ghi các giá trị này; việc biên dịch với `F_CPU` không tự thay đổi fuse phần cứng.

Bảng 2.5: Dung lượng bộ nhớ của chương trình (`avr-size`, avr-gcc 16.1.0, `-Os`)

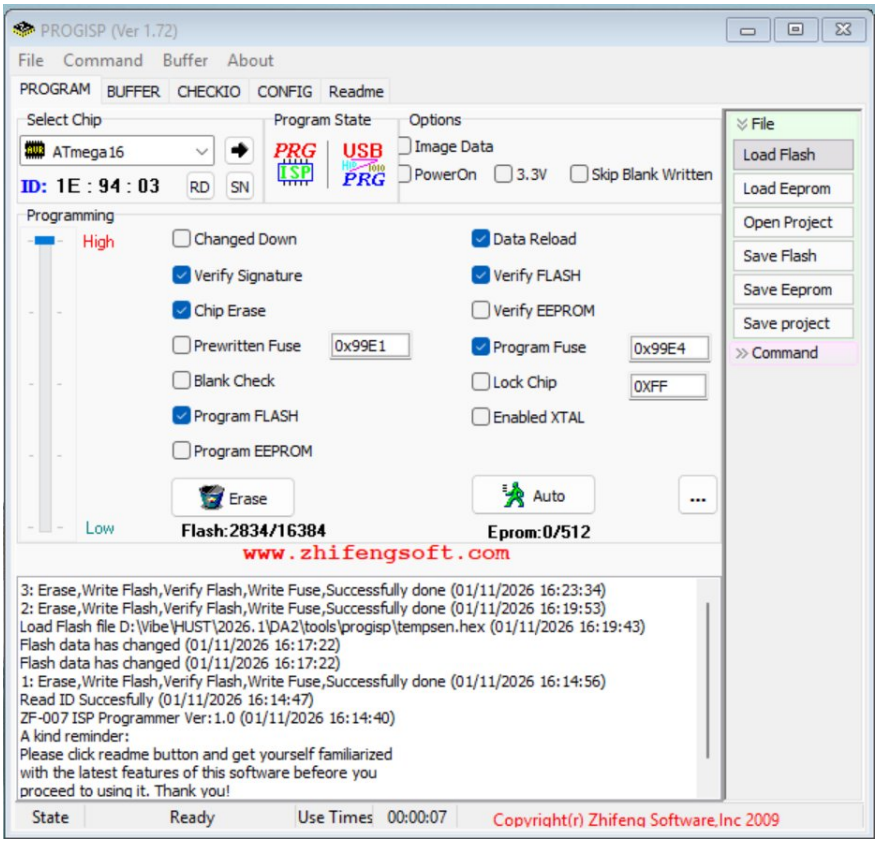
Vùng nhớ	Thành phần	Sử dụng	Dung lượng chip	Tỷ lệ
Flash	`.text` (2674 B) + `.data` (156 B)	2830 B	16384 B	17,3 %
SRAM	`.data` (156 B) + `.bss` (0 B), chưa tính ngăn xếp	156 B	1024 B	15,2 %
EEPROM	Không sử dụng	0 B	512 B	0 %

Toàn bộ 156 B của vùng `.data` là các chuỗi hằng hiển thị trên LCD (cùng chuỗi định dạng `"%05u"` của thư viện LCD); chúng được chép từ Flash sang SRAM khi khởi động. Biến cục bộ như `buffer[32]` trong `main()` và vùng nhớ tạm của `sprintf` nằm trên ngăn xếp nên không có trong số liệu này. Chương trình còn nhiều dư địa bộ nhớ cho các hướng mở rộng ở mục 4.3.

2.4.2 Quy trình nạp chương trình bằng phần mềm ProgISP

Các bước nạp chương trình vào ATmega16 được trình bày dưới đây; giao diện tham khảo ở Hình 2.5.

1. Kết nối USBasp giữa máy tính và cổng ISP trên kit.
2. Mở ProgISP 1.72.
3. Tại *Select Chip*, chọn `ATMEGA16`.
4. Nhấn RD (Read) để kiểm tra kết nối; mã nhận diện dự kiến là `1E : 94 : 03`.
5. Đặt High Fuse = `0xD9`, Low Fuse = `0xE4` (mã `0xD9E4`); bật tùy chọn *Program Fuse*.
6. Nhấn *Load Flash*, chọn tệp `src/tempsen.hex`.
7. Nhấn *Auto* để xóa chip, ghi và kiểm tra Flash, ghi fuse. Khi báo *Successfully done*, vi điều khiển khởi động lại và chạy chương trình.



Hình 2.5: Giao diện ProgISP khi nạp firmware và cấu hình fuse `0xD9E4`

CHƯƠNG 3

KẾT QUẢ THỰC NGHIỆM

3.1 Kết quả thực nghiệm trên phần cứng

Hệ thống đo nhiệt độ và cảnh báo ngưỡng sử dụng vi điều khiển ATmega16 và cảm biến DHT11 đã được lắp ráp, nạp mã thực thi và kiểm thử thực tế trên Kit phát triển AVR tại phòng thí nghiệm Trường Điện – Điện tử, Đại học Bách khoa Hà Nội. Các trạng thái chức năng ghi nhận trong quá trình thử nghiệm được tổng hợp dưới đây; phạm vi và giới hạn của phép kiểm chứng được trình bày ở mục 3.2.2.

3.1.1 Giai đoạn khởi động và màn hình chào mừng

Khi vừa cấp nguồn nuôi 5V từ cổng nạp USB, vi điều khiển hoàn tất quá trình reset phần cứng, tắt cổng JTAG nội, thiết lập chiều cổng I/O và cấu hình màn hình LCD. Sau khi khởi tạo, màn hình LCD 1602 hiển thị thông điệp chào mừng khởi động:

Dòng 1	`DA2: DO NHIET DO`
Dòng 2	`DHT11 SENSOR`

Màn hình duy trì thông điệp này trong khoảng 1,5 giây theo trễ phần mềm. Trong suốt giai đoạn khởi động, toàn bộ 5 đèn LED cảnh báo trên các chân PD0 – PD4 đều tắt nhờ lệnh khởi tạo kéo mức 1 (`PORTD |= 0x1F`). Sau 1,5 giây, lệnh xóa màn hình (`0x01`) được phát ra để chuẩn bị cho chu trình hiển thị dữ liệu đo lường.

3.1.2 Trạng thái đo lường bình thường ($T < 30°C$)

Ở trạng thái không cảnh báo, giá trị được ghi nhận trên LCD là 26.5°C (nhỏ hơn ngưỡng cài đặt 30°C):

- Cảm biến DHT11 thu thập dữ liệu và gửi gói tin 40-bit về MCU. Quá trình kiểm tra tính toàn vẹn dữ liệu bằng mã Checksum thành công, dữ liệu nhận được hợp lệ.

- Màn hình LCD 1602 hiển thị thông số:

Dòng 1	`Temp: 26.5 °C`
Dòng 2	`HIGH Th: 30°C`

- Do giá trị nhiệt độ nhận được nhỏ hơn ngưỡng cài đặt, dòng 2 hiển thị ngưỡng `"HIGH Th: 30°C"` mà không xuất hiện bất kỳ dấu hiệu cảnh báo nào.
- Dàn 5 LED cảnh báo nối với các chân PD0 – PD4 tiếp tục được giữ ở mức logic 1, nghĩa là tắt.

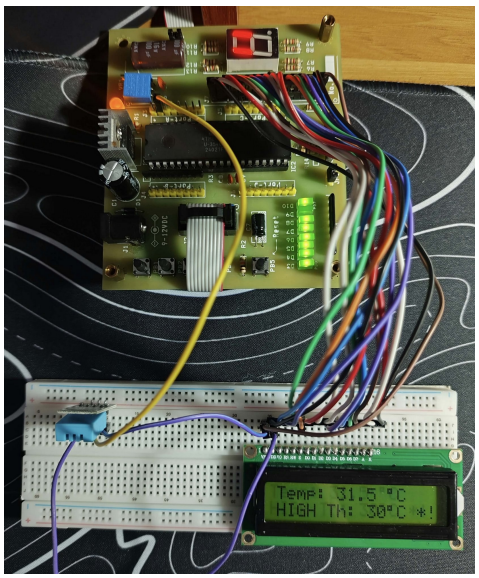
3.1.3 Trạng thái cảnh báo vượt ngưỡng quá nhiệt ($T \geq 30°C$)

Để kiểm thử phản ứng của hệ thống khi có sự cố quá nhiệt, người thực hiện sử dụng nguồn nhiệt gián tiếp (hơi ấm nhẹ áp gần bề mặt cảm biến DHT11). Khi giá trị nhiệt độ nhận được đạt hoặc vượt 30.0°C:

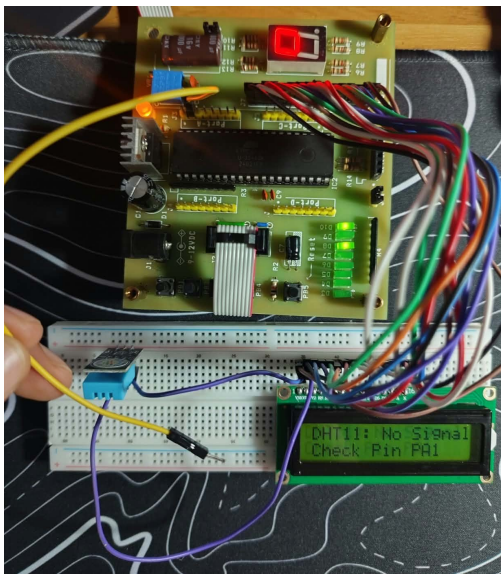
- Vi điều khiển phát hiện điều kiện *temp_int* ≥ 30°C ở chu kỳ đọc hợp lệ tiếp theo (mỗi vòng lặp có trễ phần mềm 2 giây, cộng thời gian đọc và hiển thị).
- Màn hình LCD cập nhật trạng thái báo động tương ứng:

Dòng 1	`Temp: 31.5 °C`
Dòng 2	`HIGH Th: 30°C *!`

  Trên dòng 2 xuất hiện chuỗi ký hiệu báo động `"*!"` để báo cho người vận hành.
- Đồng thời, phép toán che mặt nạ `PORTD &= 0xE0` kéo 5 chân PD0 – PD4 xuống mức 0, làm 5 LED cảnh báo cùng sáng.
- Khi nguồn nhiệt được rút ra và giá trị nhiệt độ nhận được giảm trở lại dưới 30.0°C, màn hình tự động xóa ký hiệu `"*!"` và dàn đèn LED tự động tắt, trả hệ thống về trạng thái giám sát bình thường mà không cần thao tác reset thủ công.



(a) Màn hình LCD khi cảnh báo quá nhiệt (31.5°C)



(b) Màn hình LCD khi không có tín hiệu từ cảm biến

Hình 3.1: Hình ảnh thực tế hệ thống vận hành trong các kịch bản thử nghiệm

Ghi chú: Dãy LED trên kit gồm 8 LED nối PD0–PD7 (từ D3 ở dưới lên D10 ở trên). Ba LED trên cùng (PD5–PD7) dùng chung đường RW, RS, EN của LCD nên sáng theo mức logic của các tín hiệu này, không thuộc khối cảnh báo. Ở (a), 5 LED cảnh báo PD0–PD4 sáng; ở (b), 5 LED này tắt.

3.1.4 Kiểm tra phản hồi khi mất cảm biến và xử lý lỗi hệ thống

Nhằm kiểm tra độ tin cậy và khả năng tự bảo vệ của hệ thống trước các tình huống bất thường, nhóm đã tiến hành xác minh theo hai cấp độ:

- Thử nghiệm thực tế khi mất cảm biến (Kiểm chứng phần cứng): Để mô phỏng sự cố tuột dây tín hiệu, nhóm chủ động rút chân DATA (PA1). Vòng chờ đầu tiên của `DHT11_Read` hết hạn và hàm trả về −1 sau khoảng 29 ms (Bảng 2.4); vi điều khiển không bị treo. Màn hình LCD cập nhật thông báo chẩn đoán lỗi sau khi hàm đọc cảm biến phát hiện timeout (Hình 3.1b):

Dòng 1	`DHT11: No Signal`
Dòng 2	`Check Pin PA1`

- Kiểm chứng giải thuật xử lý Checksum (Phân tích mã nguồn): Đối với trường hợp đường truyền bị can nhiễu, do chưa có thiết bị tạo xung nhiễu chuyên dụng, nhánh xử lý sai Checksum hiện chỉ được đối chiếu với luồng logic mã nguồn (nhánh `status == -6`). Thuật toán nhận diện sự sai lệch tổng kiểm tra và hiển thị thông báo chuẩn 16 ký tự:

Dòng 1	`DHT11: Sum Error`
Dòng 2	`Checksum Invalid`

- Trạng thái xử lý lỗi xác định: Trong cả hai tình huống, dàn 5 LED được đưa về trạng thái tắt (`PORTD |= 0x1F`) như phân tích ở mục 2.3.1.

Bảng 3.1: Bảng tổng hợp các kịch bản kiểm tra và kết quả đánh giá hệ thống

Kịch bản kiểm tra	Phương pháp xác minh	Giá trị LCD	Nội dung hiển thị trên LCD 1602	Trạng thái dàn LED
Khởi động hệ thống	Thực nghiệm trên kit	–	Dòng 1: `DA2: DO NHIET DO` Dòng 2: `DHT11 SENSOR`	Tắt toàn bộ
Nhiệt độ bình thường	Thực nghiệm trên kit	26.5°C	Dòng 1: `Temp: 26.5 °C` Dòng 2: `HIGH Th: 30°C`	Tắt toàn bộ
Nhiệt độ vượt ngưỡng	Thực nghiệm trên kit	31.5°C	Dòng 1: `Temp: 31.5 °C` Dòng 2: `HIGH Th: 30°C *!`	Bật sáng cả 5 LED
Mất tín hiệu cảm biến	Thực nghiệm trên Kit (Rút chân PA1)	–	Dòng 1: `DHT11: No Signal` Dòng 2: `Check Pin PA1`	Tắt toàn bộ (Xác định)
Sai mã Checksum	Phân tích giải thuật mã nguồn	–	Dòng 1: `DHT11: Sum Error` Dòng 2: `Checksum Invalid`	Tắt toàn bộ (Xác định)

3.2 Đánh giá kết quả và phạm vi kiểm chứng

3.2.1 Kết quả chức năng đã ghi nhận

Các mô tả và ảnh ở mục 3.1 cho thấy chương trình có thể hiển thị giá trị nhiệt độ, báo vượt ngưỡng bằng LCD và LED, đồng thời hiển thị thông báo khi mất tín hiệu cảm biến. Bảng 3.1 phân biệt kịch bản thử trên phần cứng với nhánh checksum mới được xem xét qua mã nguồn. Phân tích mã nguồn cho thấy điều kiện `temp_int >= 30` bao gồm cả trường hợp đúng ngưỡng; báo cáo hiện chưa có kết quả thử riêng tại 30°C.

Hình 3.1a ghi nhận LCD hiển thị 31.5°C cùng ký hiệu `*!` và 5 LED cảnh báo sáng. Vì chương trình in trực tiếp T_{int} và T_{dec} nhận từ cảm biến, phần thập phân `.5` trên LCD chính là giá trị Byte 3 của gói tin: module đang dùng trả về byte thập phân khác 0, khác

với bản tài liệu tham chiếu ghi độ phân giải 1°C [2]. Các giá trị này phản ánh dữ liệu cảm biến gửi về; độ chính xác so với nhiệt độ môi trường cần được đối chiếu thêm với nhiệt kế tham chiếu.

Checksum chỉ xác nhận gói tin được truyền đúng, không xác nhận độ chính xác vật lý của phép đo. Khi có lỗi, chương trình tắt 5 LED và hiển thị chẩn đoán; vì vậy LED tắt không đủ để kết luận môi trường đang dưới ngưỡng.

3.2.2 Giới hạn của kết quả thực nghiệm

Báo cáo hiện chưa có các dữ liệu sau để đánh giá định lượng:

- Các cặp số đo đồng thời giữa hệ thống và nhiệt kế tham chiếu tại nhiều mức nhiệt, nên chưa xác định được sai lệch trung bình hoặc sai lệch lớn nhất.
- Nhật ký chạy liên tục gồm tổng số lần đọc, số lần hợp lệ, số lỗi timeout và checksum, nên chưa tính được tỷ lệ đọc thành công hoặc độ ổn định dài hạn.
- Thời gian từ khi số đo chạm ngưỡng đến khi LED đổi trạng thái; trễ 2 giây trong mã nguồn không thể thay thế phép đo độ trễ toàn hệ thống.
- Thử lặp lại tại dưới ngưỡng, đúng ngưỡng và trên ngưỡng, cũng như thao tác rút rồi nối lại cảm biến khi hệ thống đang cảnh báo.

Kết quả hiện có hỗ trợ đánh giá chức năng cơ bản, chưa xác nhận độ chính xác đo hoặc độ tin cậy dài hạn.

CHƯƠNG 4

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

4.1 Kết luận

Đề tài đã triển khai mô hình đo nhiệt độ và cảnh báo ngưỡng trên nền tảng ATmega16, DHT11 và LCD 1602. Các kết quả chính gồm:

- Vẽ lại sơ đồ nguyên lý từ tài liệu giảng viên cung cấp và thiết kế PCB trên Altium Designer; xác định kết nối cảm biến ngoài kit và các nhóm chân dùng cho LCD, LED.
- Triển khai chương trình C đọc gói dữ liệu 40-bit từ DHT11, kiểm tra checksum, giới hạn mỗi vòng chờ bằng timeout 7,5–8,75 ms và phân loại thông báo lỗi. Chương trình chiếm 2830 B (17,3 %) bộ nhớ Flash.
- Cấu hình thư viện LCD giao tiếp 4-bit và mặt nạ bit để cập nhật LCD, LED trên các nhóm chân dùng chung cổng D.
- Thống nhất cấu hình RC nội 8 MHz, fuse `0xD9E4`, giá trị `F_CPU` và thao tác vô hiệu hóa JTAG nhằm sử dụng PC4, PC5 cho LCD.
- Ghi nhận các trạng thái hiển thị nhiệt độ, cảnh báo ngưỡng và báo mất tín hiệu cảm biến như Chương 3. Nhánh lỗi checksum hiện được đối chiếu bằng phân tích mã nguồn.

Mô hình đã thực hiện đúng các chức năng đề ra trong phạm vi thử nghiệm đã trình bày; các hạn chế còn lại được tổng hợp ở mục 4.2.

4.2 Hạn chế của đề tài

Các hạn chế hiện tại của mô hình gồm:

- Cảm biến DHT11 có dải đo nhiệt độ hẹp (0°C – 50°C) và sai số tương đối lớn ($\pm$2.0°C), chu kỳ lấy mẫu tương đối chậm (chương trình chèn trễ 2 giây sau mỗi vòng lặp) nên chưa phù hợp cho các môi trường đòi hỏi đáp ứng thời gian thực cực nhanh.

- Chưa có phép đo đối chiếu với nhiệt kế tham chiếu, nhật ký chạy dài hạn và phép đo độ trễ cảnh báo, nên độ chính xác và độ ổn định của hệ thống mới được đánh giá ở mức chức năng (mục 3.2.2).
- Chưa có hysteresis, nên cảnh báo có thể đổi trạng thái nhiều lần khi số đo dao động quanh ngưỡng.
- Ngưỡng cảnh báo nhiệt độ (30°C) hiện đang được cố định cứng trong mã nguồn, người sử dụng chưa thể linh hoạt thay đổi ngưỡng trực tiếp từ bên ngoài bằng phím bấm mà phải biên dịch lại chương trình.
- Cơ chế xử lý khi mất cảm biến hiện đưa toàn bộ LED về trạng thái tắt để tránh báo động giả từ dữ liệu cũ lỗi thời, nhưng có nhược điểm là không tiếp tục cảnh báo bằng đèn nếu môi trường thực tế vẫn đang quá nhiệt.
- Hệ thống chưa có bộ nhớ lưu trữ lịch sử biến thiên nhiệt độ theo thời gian và chưa có khả năng truyền thông dữ liệu từ xa lên mạng máy tính.

4.3 Hướng phát triển mở rộng

Từ mô hình phần cứng và phần mềm hiện có, hệ thống có thể tiếp tục được nghiên cứu và nâng cấp theo các hướng ứng dụng thực tế:

1. Hoàn thiện kiểm chứng: thu log dữ liệu thô, đối chiếu nhiệt kế tham chiếu, thống kê lỗi khi chạy liên tục và đo độ trễ. Bổ sung hysteresis với ngưỡng bật/tắt riêng theo yêu cầu ứng dụng và độ biến động của số đo.
2. Nâng cấp cảm biến: Thay thế cảm biến DHT11 bằng các dòng cảm biến kỹ thuật số có độ chính xác cao hơn phù hợp với yêu cầu ứng dụng; các phương án có thể khảo sát gồm DHT22, cảm biến I2C như Sensirion SHT30/SHT35 hoặc Bosch BME280 (đo đồng thời nhiệt độ, độ ẩm và áp suất khí quyển).
3. Bổ sung giao diện người dùng tương tác linh hoạt: Tích hợp bàn phím ma trận 4 × 4 hoặc cụm nút bấm tăng/giảm/cài đặt kết hợp còi báo, cho phép người dùng tự do cài đặt ngưỡng nhiệt độ cảnh báo và lưu trữ giá trị cấu hình này vào bộ nhớ không biến động EEPROM của ATmega16.
4. Bổ sung chế độ cảnh báo lỗi cảm biến chuyên biệt: Lập trình cơ chế nhấp nháy toàn bộ dàn LED (tần số 2 Hz) hoặc còi buzzer ngắt quãng khi mất kết nối cảm

biến để người vận hành phân biệt rõ ràng giữa trạng thái ”Bình thường” (LED tắt), ”Cảnh báo quá nhiệt” (LED sáng liên tục) và ”Lỗi cảm biến” (LED nhấp nháy).

5. Tích hợp kết nối không dây và phát triển ứng dụng IoT: Kết nối cổng giao tiếp nối tiếp USART của ATmega16 với các mô-đun truyền thông không dây như Wi-Fi ESP8266/ESP32, Bluetooth HC-05 hoặc chuẩn truyền xa LoRa để gửi dữ liệu nhiệt độ lên máy chủ đám mây qua giao thức MQTT/HTTP. Qua đó, người quản lý có thể giám sát biểu đồ nhiệt độ môi trường và nhận thông báo cảnh báo khẩn cấp trực tiếp trên ứng dụng điện thoại thông minh (Blynk, ThingsBoard) qua kết nối mạng. Khi dùng USART phần cứng, cần bố trí lại LED trên PD0/RXD và PD1/TXD để tránh xung đột chức năng, đồng thời kiểm tra mức điện áp giao tiếp của module.

TÀI LIỆU THAM KHẢO

[1] Atmel Corporation, “ATmega16/ATmega16L Datasheet”, Rev. 2466T–AVR–07/10, 2010. https://ww1.microchip.com/downloads/en/DeviceDoc/doc2466.pdf, truy cập ngày 01/11/2026.

[2] Aosong Electronics, “DHT11 Digital Temperature and Humidity Sensor” (bản tài liệu tiếng Trung, 6 trang, không ghi năm), mục 1, 4 và 6. Bản lưu: https://cdn-shop.adafruit.com/datasheets/DHT11-chinese.pdf, truy cập ngày 01/11/2026.

[3] Texas Instruments, “LM35 Precision Centigrade Temperature Sensors”, SNIS159H, 08/1999, sửa đổi 12/2017. https://www.ti.com/lit/ds/symlink/lm35.pdf, truy cập ngày 02/11/2026.

[4] Dallas Semiconductor (nay thuộc Analog Devices), “DS18B20 Programmable Resolution 1-Wire Digital Thermometer”. Bản lưu: https://cdn.sparkfun.com/datasheets/Sensors/Temp/DS18B20.pdf, truy cập ngày 02/11/2026.

[5] Hitachi, “HD44780U (LCD-II): Dot Matrix Liquid Crystal Display Controller/Driver”, ADE-207-272(Z), Rev. 0.0, 09/1999. Bản lưu: https://cdn.sparkfun.com/assets/9/5/f/7/b/HD44780.pdf, truy cập ngày 01/11/2026.

[6] “Sơ đồ mạch Kit”, tài liệu nội bộ do giảng viên cung cấp phục vụ học phần Đồ án Thiết kế II; tệp lưu trong hồ sơ đồ án tại `docs/ref`. Không ghi tác giả và năm xuất bản. Nhóm sử dụng làm cơ sở vẽ lại sơ đồ và thiết kế PCB.

[7] AVR-LibC Project, “AVR-LibC User Manual: delay.h File Reference”. https://avrdudes.github.io/avr-libc/avr-libc-user-manual/delay_8h.html, truy cập ngày 01/11/2026.

PHỤ LỤC: TOÀN BỘ MÃ NGUỒN CHƯƠNG TRÌNH

Phụ lục A: File mã nguồn chính `tempsen.c`

```
#define F_CPU 8000000UL
#include <avr/io.h>
#include <util/delay.h>
#include <stdio.h>
#include "hunget_lcd.h" // Thư viện LCD chuẩn Kit BK

// ======================================================
// CẤU HÌNH CHÂN DHT11 NỐI VÀO CHÂN PA1 TRÊN PORT A
// ======================================================
#define DHT_DDR   DDRA
#define DHT_PORT  PORTA
#define DHT_PIN_R PINA
#define DHT_BIT   PA1

// Hàm đọc dữ liệu từ cảm biến số DHT11
// Trả về 0 nếu thành công, < 0 nếu lỗi/timeout
int8_t DHT11_Read(uint8_t *temp_int, uint8_t *temp_dec, uint8_t *humi_int) {
    uint8_t data[5] = {0, 0, 0, 0, 0};
    uint16_t timeout;

    // 1. MCU gửi tín hiệu Start (kéo LOW tối thiểu 18ms)
    DHT_DDR |= (1 << DHT_BIT);     // Set PA1 là Output
    DHT_PORT &= ~(1 << DHT_BIT);   // Kéo xuống LOW
    _delay_ms(20);                 // Giữ mức LOW 20ms

    // 2. Nhả bus sang Input và kích hoạt trở kéo lên để chờ DHT11 phản hồi
     ↪  (tránh xung đột bus)
    DHT_DDR &= ~(1 << DHT_BIT);    // Chuyển PA1 thành Input
    DHT_PORT |= (1 << DHT_BIT);    // Bật điện trở kéo lên nội
    _delay_us(30);                 // Chờ 20-40us cho DHT11 bắt đầu phản hồi

    // Đợi DHT11 kéo LOW (khoảng 20-40µs)
    timeout = 10000;
    while (DHT_PIN_R & (1 << DHT_BIT)) {
        if (--timeout == 0) return -1; // Timeout chờ DHT11 phản hồi
    }

    // Đợi DHT11 kéo HIGH (khoảng 80µs)
    timeout = 10000;
    while (!(DHT_PIN_R & (1 << DHT_BIT))) {
        if (--timeout == 0) return -2;
    }

    // Đợi DHT11 kéo LOW chuẩn bị truyền bit đầu tiên (khoảng 80µs)
    timeout = 10000;
    while (DHT_PIN_R & (1 << DHT_BIT)) {
        if (--timeout == 0) return -3;
```

```
    }

    // 3. Đọc 40 bit (5 byte: Humi_int, Humi_dec, Temp_int, Temp_dec, Checksum)
    for (uint8_t i = 0; i < 5; i++) {
        for (int8_t j = 7; j >= 0; j--) {
            // Đợi hết xung LOW 50µs của từng bit
            timeout = 10000;
            while (!(DHT_PIN_R & (1 << DHT_BIT))) {
                if (--timeout == 0) return -4;
            }

            // Đo độ rộng xung HIGH:
            // 26-28µs -> bit 0
            // 70µs    -> bit 1
            // Chờ 38µs rồi đọc trạng thái chân
            _delay_us(38);
            if (DHT_PIN_R & (1 << DHT_BIT)) {
                data[i] |= (1 << j);
                // Đợi xung HIGH kết thúc
                timeout = 10000;
                while (DHT_PIN_R & (1 << DHT_BIT)) {
                    if (--timeout == 0) return -5;
                }
            }
        }
    }

    // 4. Kiểm tra mã kiểm tra lỗi (Checksum 8-bit)
    if ((uint8_t)(data[0] + data[1] + data[2] + data[3]) != data[4]) {
        return -6; // Sai checksum
    }

    *humi_int  = data[0];
    *temp_int  = data[2];
    *temp_dec  = data[3];
    return 0; // Đọc thành công
}

int main(void) {
    // 1. TẮT JTAG (BẮT BUỘC để giải phóng PORT C điều khiển LCD)
    MCUCSR = (1 << JTD);
    MCUCSR = (1 << JTD);

    // 2. KHỞI TẠO LCD
    LCD4_INIT(0, 0);

    // 3. KHỞI TẠO DÃY LED CẢNH BÁO (PD0 - PD4)
    // PD5-PD7 là Output cho LCD (đã set trong thư viện LCD)
    // Set thêm PD0-PD4 là Output cho LED
    DDRD |= 0x1F;  // Set 5 chân thấp là Output
    PORTD |= 0x1F; // Xuất mức 1 để TẮT hết LED ban đầu (Active Low)

    // --- MÀN HÌNH CHÀO MỪNG ---
    LCD4_CUR_GOTO(1, 0);
```

```
LCD4_OUT_STR("DA2: DO NHIET DO");
LCD4_CUR_GOTO(2, 2);
LCD4_OUT_STR("DHT11 SENSOR");
_delay_ms(1500);
LCD4_OUT_CMD(0x01); // Xóa màn hình
_delay_ms(2);

char buffer[32];
uint8_t temp_int = 0;
uint8_t temp_dec = 0;
uint8_t humi_int = 0;

while (1) {
    int8_t status = DHT11_Read(&temp_int, &temp_dec, &humi_int);

    if (status == 0) {
        // --- ĐỌC THÀNH CÔNG TỪ DHT11 ---
        // Dòng 1: "Temp: 31.5 °C   " (đúng 16 ký tự chuẩn)
        LCD4_CUR_GOTO(1, 0);
        sprintf(buffer, "Temp: %u.%u \xDF""C   ", temp_int, temp_dec);
        LCD4_OUT_STR(buffer);

        // Dòng 2: Hiển thị ngưỡng cảnh báo "HIGH Th: 30°C"
        LCD4_CUR_GOTO(2, 0);
        if (temp_int >= 30) {
            // Nhiệt độ vượt ngưỡng 30 độ C: Báo WARNING và BẬT LED (đúng 16
             ↪ ký tự)
            LCD4_OUT_STR("HIGH Th: 30\xDF""C *!");
            PORTD &= 0xE0; // Bật sáng 5 LED cảnh báo (Active LOW)
        } else {
            // Nhiệt độ bình thường < 30 độ C: TẮT LED (đúng 16 ký tự, xóa
             ↪ dấu *!)
            LCD4_OUT_STR("HIGH Th: 30\xDF""C   ");
            PORTD |= 0x1F; // Tắt 5 LED cảnh báo
        }
    } else if (status == -6) {
        // Lỗi kiểm tra toàn vẹn dữ liệu (Checksum Error)
        LCD4_CUR_GOTO(1, 0);
        LCD4_OUT_STR("DHT11: Sum Error");
        LCD4_CUR_GOTO(2, 0);
        LCD4_OUT_STR("Checksum Invalid");
        PORTD |= 0x1F; // Trạng thái xác định: Tắt LED cảnh báo
    } else {
        // Trường hợp mất tín hiệu, chưa cắm chân PA1 hoặc lỏng dây
        LCD4_CUR_GOTO(1, 0);
        LCD4_OUT_STR("DHT11: No Signal");
        LCD4_CUR_GOTO(2, 0);
        LCD4_OUT_STR("Check Pin PA1   ");
        PORTD |= 0x1F; // Trạng thái xác định: Tắt LED cảnh báo
    }

    // Chu kỳ lấy mẫu khuyến nghị theo datasheet của Aosong (2 giây)
    _delay_ms(2000);
}
```

```
    return 0;
}
```

Phụ lục B: Thư viện điều khiển LCD `hunget_lcd.h`

```
#ifndef HUNGET_LCD_H
#define HUNGET_LCD_H

#include <avr/io.h>
#include <util/delay.h>
#include <stdio.h>
#include <stdlib.h>

// ======================================================
// CẤU HÌNH CHÂN THEO ĐÚNG SƠ ĐỒ MẠCH KIT
// ======================================================
// 1. CHÂN DỮ LIỆU (DATA) -> PORT C
// Sử dụng 4 bit cao: D4->PC4, D5->PC5, D6->PC6, D7->PC7
#define LCD_DATA_DDR DDRC
#define LCD_DATA_PORT PORTC
#define LCD_DATA_PIN PINC

// 2. CHÂN ĐIỀU KHIỂN (CONTROL) -> PORT D
#define LCD_CTRL_DDR DDRD
#define LCD_CTRL_PORT PORTD

// Định nghĩa lại chính xác theo sơ đồ J7:
#define LCD_RW 5 // Pin 5 LCD nối PD5
#define LCD_RS 6 // Pin 4 LCD nối PD6
#define LCD_EN 7 // Pin 6 LCD nối PD7

// ======================================================
// CÁC HÀM GIAO TIẾP MỨC THẤP
// ======================================================
// Hàm tạo xung Enable trên PORT D
static inline void LCD_Enable(void) {
    LCD_CTRL_PORT |= (1 << LCD_EN); // Bật E (PD7)
    _delay_us(5);
    LCD_CTRL_PORT &= ~(1 << LCD_EN); // Tắt E
    _delay_us(100);
}

// Hàm gửi 4 bit dữ liệu ra PORT C
static inline void LCD_Send4Bit(unsigned char Data) {
    // 1. Gửi 4 bit cao của dữ liệu ra PORT C (vào các chân PC4-PC7)
    // Giữ nguyên trạng thái của 4 chân thấp PC0-PC3 bằng mặt nạ bit
    unsigned char current_port_c = LCD_DATA_PORT & 0x0F;
    LCD_DATA_PORT = current_port_c | (Data & 0xF0);
    // 2. Tạo xung chốt bên PORT D
    LCD_Enable();
}
```

```
// Hàm gửi lệnh (RS=0, RW=0 tại PORT D)
static inline void LCD4_OUT_CMD(unsigned char command) {
    LCD_CTRL_PORT &= ~(1 << LCD_RS);
    LCD_CTRL_PORT &= ~(1 << LCD_RW);
    LCD_Send4Bit(command); // Gửi 4 bit cao (sang PORT C)
    LCD_Send4Bit(command << 4); // Gửi 4 bit thấp (sang PORT C)
    if (command < 4) _delay_ms(2);
    else _delay_us(50);
}

// Hàm gửi dữ liệu (RS=1, RW=0 tại PORT D)
static inline void LCD4_OUT_DATA(unsigned char data) {
    LCD_CTRL_PORT |= (1 << LCD_RS);
    LCD_CTRL_PORT &= ~(1 << LCD_RW);
    LCD_Send4Bit(data);
    LCD_Send4Bit(data << 4);
    _delay_us(50);
}

// ========================================================
// CÁC HÀM NGƯỜI DÙNG (USER FUNCTIONS)
// ========================================================
static inline void LCD4_INIT(unsigned char cursor, unsigned char blink) {
    // Khởi tạo hướng dữ liệu (Output)
    // PC4-PC7 là Output (Data)
    LCD_DATA_DDR |= 0xF0;
    // PD5, PD6, PD7 là Output (Control)
    LCD_CTRL_DDR |= (1<<LCD_RS) | (1<<LCD_RW) | (1<<LCD_EN);
    // Reset mức logic các chân điều khiển về 0
    LCD_CTRL_PORT &= ~((1<<LCD_RS) | (1<<LCD_RW) | (1<<LCD_EN));
    _delay_ms(50); // Chờ LCD ổn định sau khi cấp nguồn

    // Chuỗi Reset (Gửi 0x30 ba lần theo chuẩn HD44780)
    LCD_Send4Bit(0x30); _delay_ms(5);
    LCD_Send4Bit(0x30); _delay_ms(2);
    LCD_Send4Bit(0x30); _delay_ms(2);

    // Chuyển sang chế độ 4-bit
    LCD_Send4Bit(0x20);
    _delay_ms(2);

    // Cấu hình hiển thị
    LCD4_OUT_CMD(0x28); // 4-bit, 2 dòng, font 5x8
    _delay_ms(1);
    LCD4_OUT_CMD(0x08); // Tắt hiển thị
    _delay_ms(1);
    LCD4_OUT_CMD(0x01); // Xóa màn hình
    _delay_ms(3);

    unsigned char cmd = 0x0C; // Mặc định bật màn hình
    if(cursor) cmd |= 0x02;
    if(blink) cmd |= 0x01;
    LCD4_OUT_CMD(cmd);
    _delay_ms(1);
    LCD4_OUT_CMD(0x06); // Tự động tăng con trỏ
```

```
    _delay_ms(1);
}

void LCD4_CUR_GOTO(unsigned char row, unsigned char col) {
    unsigned char address;
    if (row == 1) address = 0x80 + col;
    else address = 0xC0 + col;
    LCD4_OUT_CMD(address);
}

void LCD4_OUT_STR(char *str) {
    while (*str) LCD4_OUT_DATA(*str++);
}

void LCD4_OUT_DEC(unsigned int num, unsigned char len) {
    char buffer[10];
    sprintf(buffer, "%05u", num);
    char *ptr = &buffer[5 - len];
    LCD4_OUT_STR(ptr);
}

void LCD4_DIS_SHIFT(unsigned char direct, unsigned char step) {
    unsigned char i;
    if (direct == 0) // Dịch trái
        for(i=0; i<step; i++) LCD4_OUT_CMD(0x18);
    else // Dịch phải
        for(i=0; i<step; i++) LCD4_OUT_CMD(0x1C);
}

#endif // HUNGET_LCD_H
```